Flow Control Methods

Transmission delay (T_t) channel

sender

time taken to put packet from server to link (T_t)

T_t is associated with packets.

Larger packets ↑ T_t ↑

Say Bandwidth = 1 bps

Length = 10 bits

1s → 1 bit

10s → 10 bit

$$T_t = \frac{\text{length}}{BW}$$

say $L = 1000$ bits $BW = 1$ Kbps

$$T_t = \frac{L}{BW} = \frac{1000\,b}{1\,K\,bps} = 1\,s$$

say $L = 1$ Kb $BW = 1$ Kbps

$$T_t = \frac{L}{BW} = \frac{1\,Kb}{1\,Kbps}$$ X

For data, K is 1024 for BW 10^3

M $(1024)^2$ 10^6

G $(1024)^3$ $(10)^9$

BW is basically signal. In Physics we deal with powers of 10.

$$T_t = \frac{1024}{1000} = 1.024 \text{ s}$$

Propogation delay: (T_p)

Time for 1 bit to go from one end of channel to other

$$T_p = \frac{d}{v} \quad \begin{matrix} \leftarrow \text{distance} \\ \leftarrow \text{velocity} \end{matrix}$$

optical fibre uses light (3×10^7 m/s).
It practically is 70% of that.

$$= 0.7 \times 3 \times 10^7$$

$$= 2.1 \times 10^8$$

use $\boxed{2 \times 10^8}$ as it is easy for calculation

say $d = 2.1$ Km, $v = 2.1 \times 10^8$ m/s

$$T_p = \frac{2.1 \times 10^3 \text{ m}}{2.1 \times 10^8 \text{ m/s}} = 10^{-5} \text{ s}$$

$$= 10^{-5} \times \frac{10^{-3}}{10^{-3}}$$

$$= 10^{-2} \text{ ms}$$

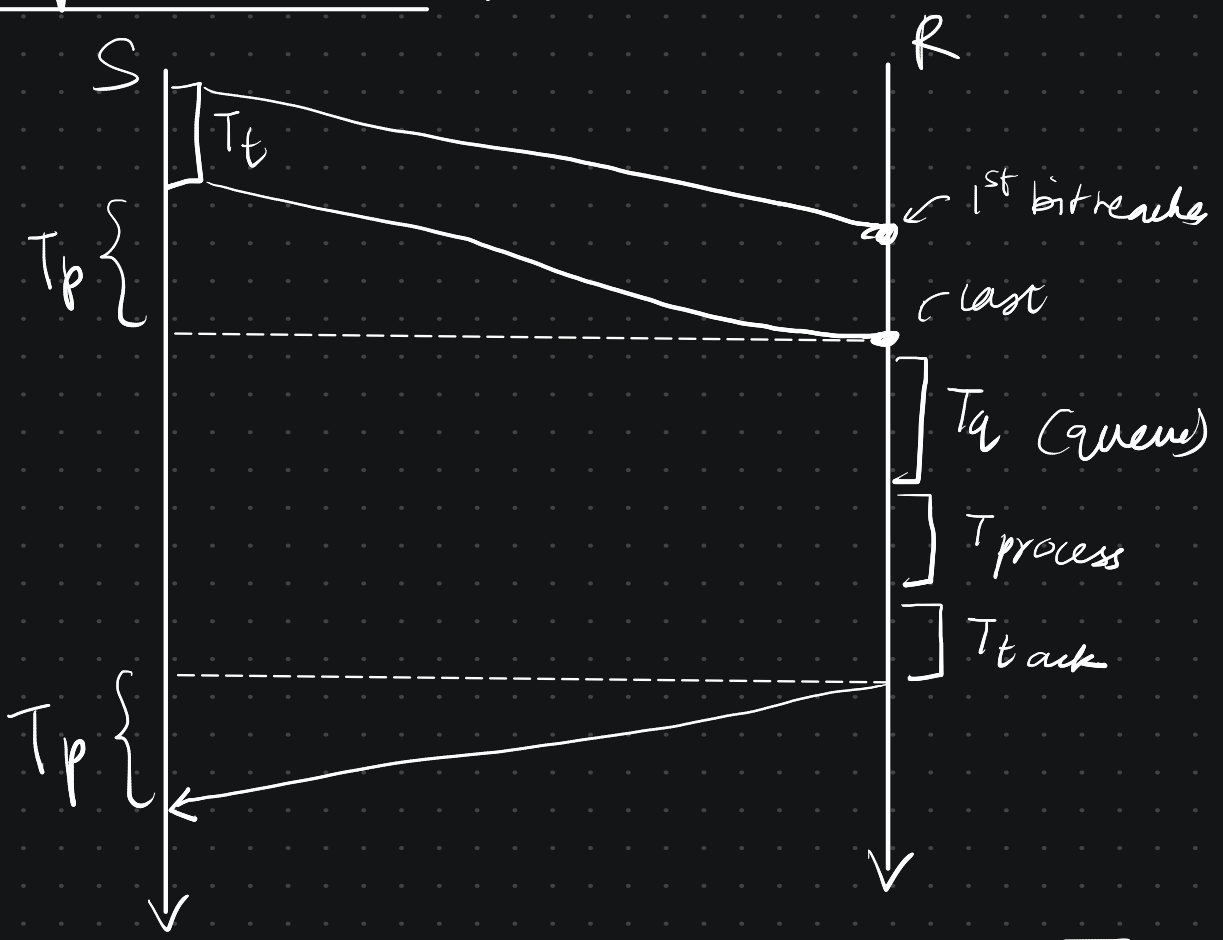
$$\text{or} = \frac{10^{-5}}{10^{-6}} \times \underbrace{10^{-6}}_{\mu} = 10 \ \mu\text{s}$$

At receiving end, packet goes into queue.

There's a queueing delay.

There's a processing delay at receiver as well.

STOP AND WAIT



$$\{\text{total time}\} = T_{t\,data} + T_q + T_{proc} + T_{t\,ack} + 2 * T_p$$

$2 * T_p$ = RTT (round trip time)

$$TT = T_{t\,data} + T_{t\,ack} + 2 * T_p$$

$T_{t\,data} = \frac{L_{data}}{BW}$ $\quad$ $T_{t\,ack} = \frac{L_{ack}}{BW}$

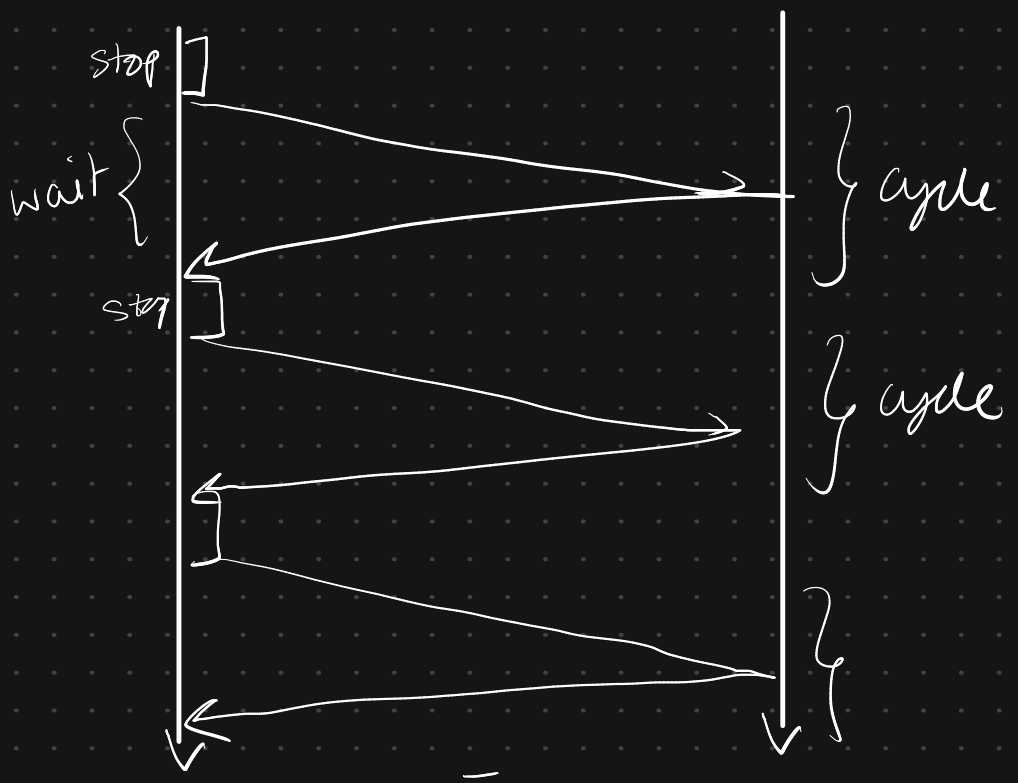
$L_{data} \gg L_{ack}$

so if not given then ignore.
consider if really big.

we don't have formulas for T_{proc}, T_q so ffk them

simplified, $TT = T_t + 2 * T_p$

called stop & wait



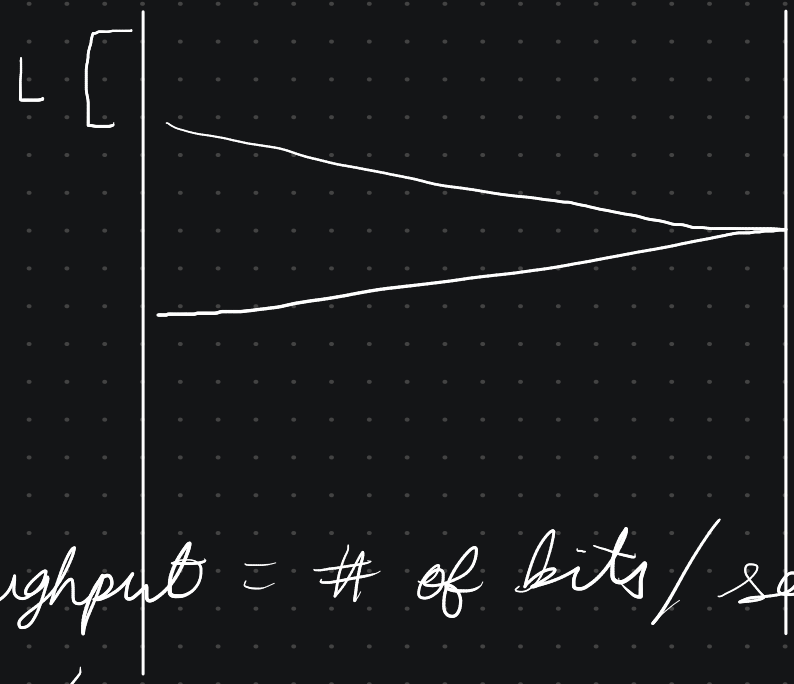
$$\eta = \frac{\text{useful time}}{\text{total time}} = \frac{t_t}{T_t + 2T_p}$$

η efficiency

$$= \frac{1}{1 + 2\frac{T_p}{T_t}}$$

$\frac{T_p}{T_t}$ — a

$$\eta = \frac{1}{1 + 2a}$$



Throughput = # of bits / second

In $(T_t + 2T_p)$ secs we send L bits (length of 1 packet)

$$\therefore \text{Throughput} = \frac{L}{T_t + 2T_p}$$

$$= \frac{L}{T_t + 2T_p} \times \frac{B}{B}$$ (B ← bandwidth)

$$= \frac{T_t \cdot B}{T_t + 2T_p}$$

$$= \frac{B}{1 + 2\frac{T_p}{T_t}}$$

$$= \frac{B}{1 + 2a} = \frac{1}{1 + 2a} B$$

Throughput = $\eta * B$

Or

Effective BW

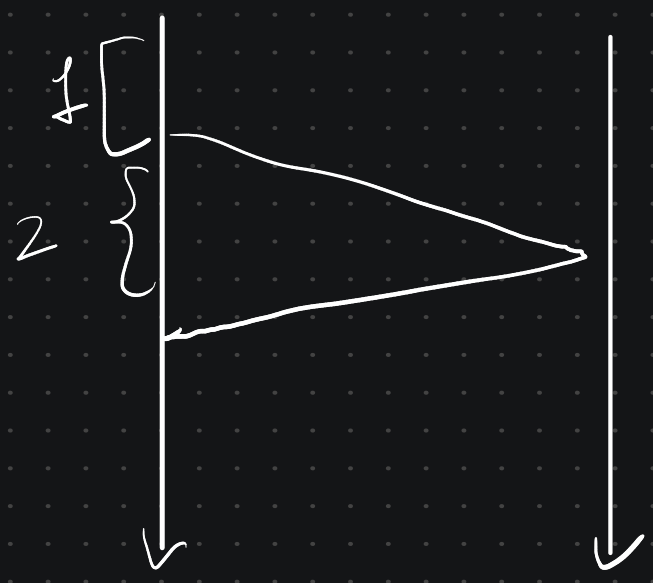
Or

BW utilizaⁿ

$T_t = 1ms$, $T_p = 1ms$ $\eta = ?$

$$a = \frac{T_p}{T_t} = \frac{1}{1} = 1$$

$$\eta = \frac{1}{1+2a} = \frac{1}{3}$$

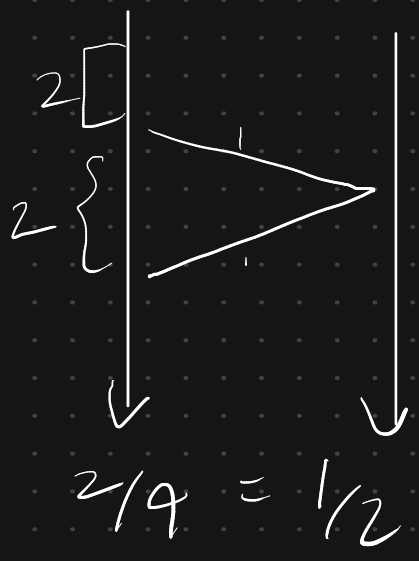


In 3ms we could've sent 3 bit we send only 1 packet

Q. $T_t = 2ms$, $T_p = 1ms$ $\eta = ?$

$$a = \frac{T_p}{T_t} = 1/2$$

$$\eta = \frac{1}{1+2a} = \frac{1}{2}.$$

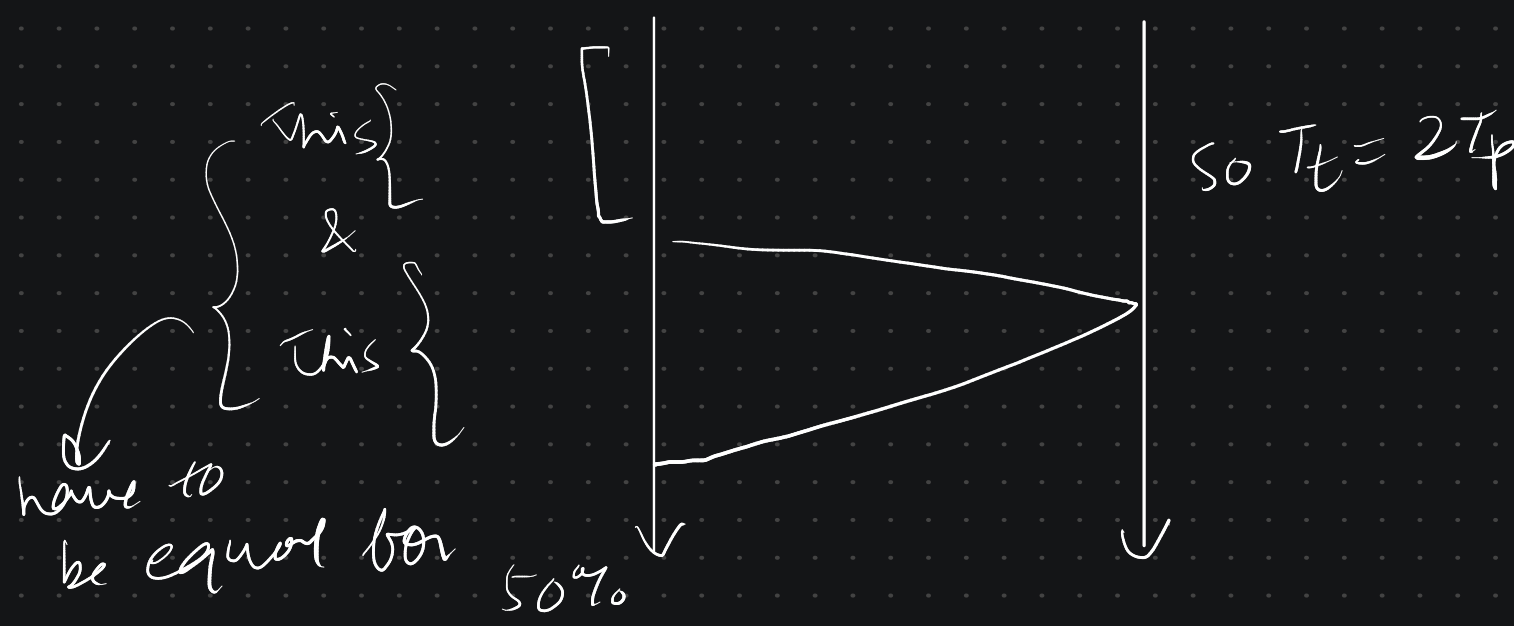


Q. What is relation bet^n T_p & T_t for $\eta = 50\%$?

$$\rightarrow \eta = \frac{1}{1+2\frac{T_p}{T_t}} \Rightarrow 0.5 = \frac{1}{1+2\frac{T_p}{T_t}}$$

$$\Rightarrow \frac{T_p}{T_t} = \frac{1}{2}$$

$$\underline{T_t = 2 * T_p}$$



If η is atleast 50%, $T_t \geq 2T_p$

What is length of packet for atleast 50% η?

$$\rightarrow \eta = \frac{1}{1+2a} = \frac{1}{2}$$

$$a = 1/2 = \frac{T_p}{T_t}$$

$$T_t \geq 2T_p$$

$$\Rightarrow \frac{L}{B} \geq 2T_p$$

$$\Rightarrow \underline{L \geq 2 \cdot T_p \cdot B}$$

$B = 4$ Mbps, $T_p = 1$ ms, $L = ?$, $\eta \geq 50\%$

$$\rightarrow \eta = \frac{1}{2} = \frac{1}{1+2a}$$

$$a \geq \frac{1}{2}$$

$$T_t \geq 2T_p$$

$$\frac{L}{B} \geq 2T_p$$

$$L \geq 2 T_p B$$

$$2 \cdot 10^{-3} \cdot 4 \cdot 10^{6}$$

$$L \geq 8 * 10^3 \text{ bits}$$

$$= 1000 \text{ Bytes}$$

* Stop and wait isn't actually used.

$$\eta = \frac{1}{1 + 2\frac{T_p}{T_t}} = \frac{1}{1 + 2 \cdot \frac{d}{V} \cdot \frac{B}{L}}$$

$$\Rightarrow \quad d \uparrow \quad \eta \downarrow$$

∴ Stop & wait is not suitable for WANs.

$$L \uparrow \quad \eta \uparrow$$

∴ For bursty data (long packets) S&W is good

Problems in stop and wait:

* If data packet is lost, receiver wouldn't know of the packet. Sender will think that packet has been sent, acknowledgment is on the way.
  So receiver waits for packet.
  Sender waits for ack.
  Deadlock.

Soln:
Sender would wait accd. to ar ACK timer.
If time finishes, then resend.

S&W + ACK → Stop & Wait ARQ
↑
Automatic Repeat Request.

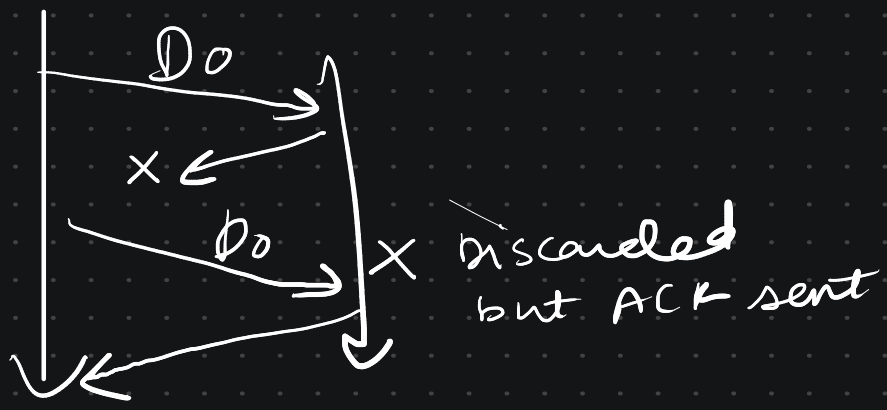
If ACK timer is too small,
no time given to get ACK.
NCA too big as time gets wasted.

Do experimentation.

* we send data, get ACK. say ACK is lost on the way. Even if data is sent, sender would think that it's a failure. sender would resend. Receiver would accept it as the second packet.

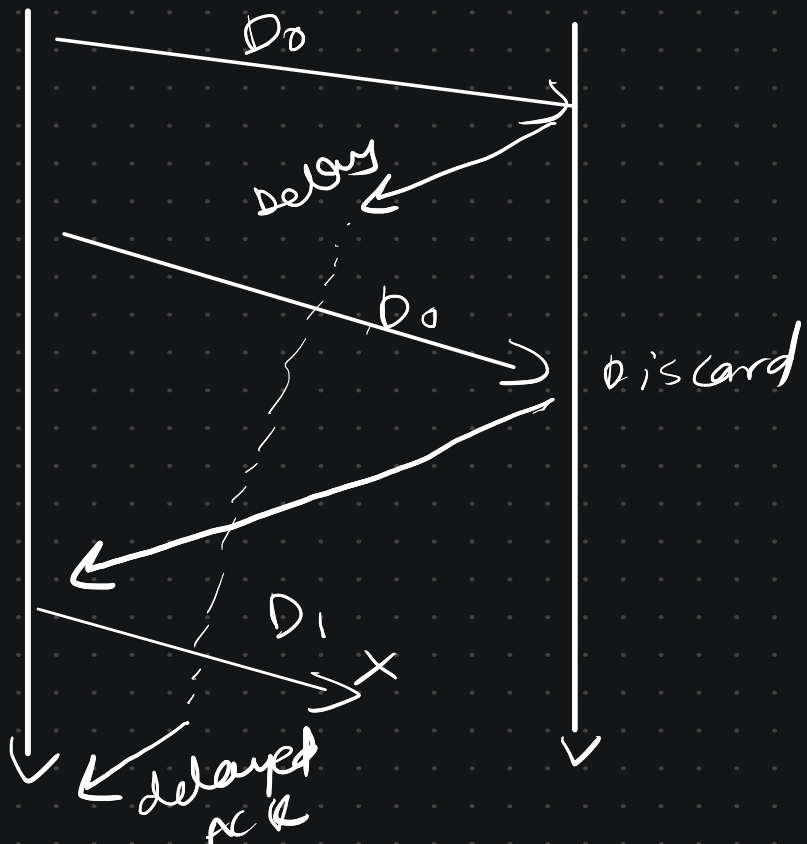
↳ Duplicate packet problem

soln: use sequence #s for data.



---

* Say ACK is delayed.

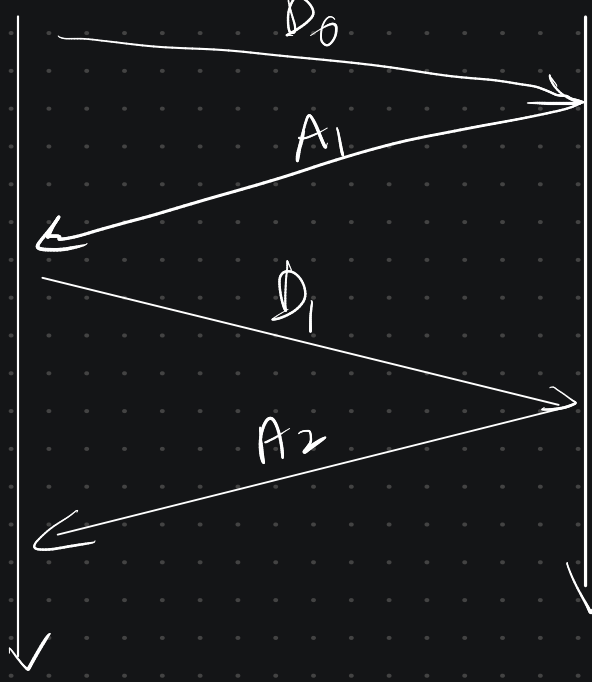


Say an ACK from packet is delayed, so resent. But it arrives after new packet D_1 is sent.
Even if D_1 fails, ACK is received.

↳ missing packet problem.

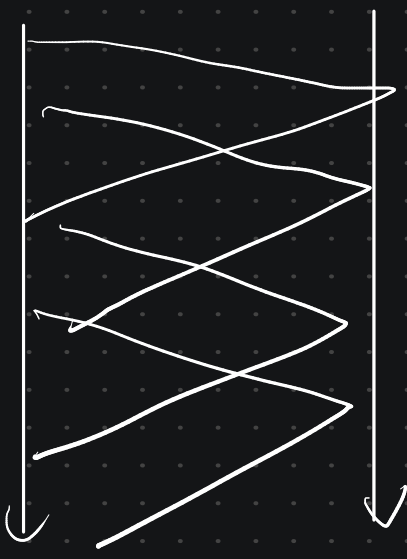
So ACKs need sequence #s as well.

ACK #s will be 1 greater ↳ represents the next packet expected



stop & wait
+
timer
+
seq. data
+
seq. ACK

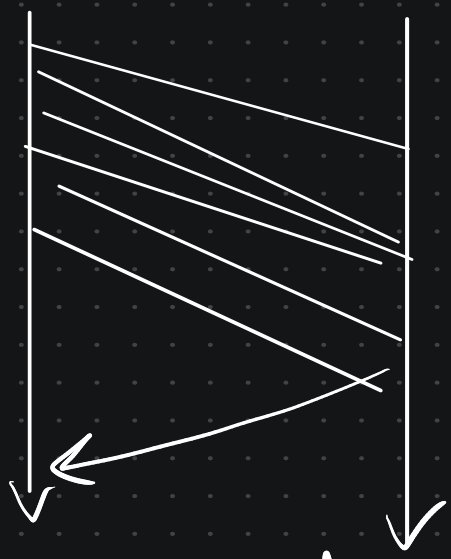
In general (not just S & W), when we send many packets, we get many ACKs.
Or we could get 1 for all together.



Independent

reliability ↑

But traffic ↑↑



Cumulative

reliability ↓:
If 1 packet is lost, have to send all.

traffic ↓

both are used by diff. algos.

Q. In S&W, 10 packets are sent from S→R. Every 4th packet is lost.

# of packets are transmitted in total?

→

1 2 3 4 4 5 6 7 7 8 9 10 10

. . . X . . . X . . . X

= 13

BTW we are talking of S→R.

Ignore ACKs

Q. S→R 400 packets to be transmitted.

channel has error prob. 0.2

Total # of transmissions?

→ $0.2 = \frac{1}{5}$

$\frac{400}{5} = 80$ $\frac{80}{5} = 16$ $\frac{16}{5} = 3.2$ $\frac{3.2}{5} = 0.64$

$400 + 80 + 16 + 3.2 + 0.64 \approx 500$

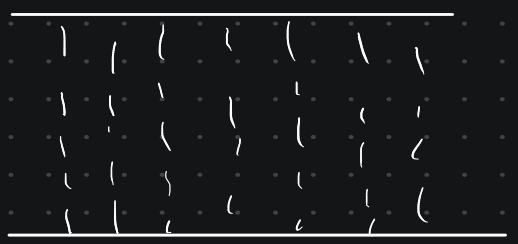
$$400 + \frac{400}{5} + \frac{400}{5^2} + \cdots$$

$$400 \times \left(\frac{1}{1 - 1/5}\right) = \frac{400 \cdot 5}{4} = 500$$

For n packets & p prob. of error:

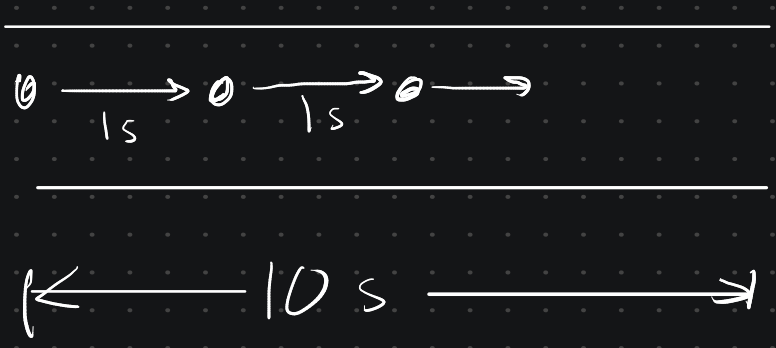
$$n + np + np^2 + \cdots = \frac{n}{1-p}$$

Capacity of a channel:



# of bits it can hold at a time.

Say B = 1. Every 1 sec we put 1 bit.



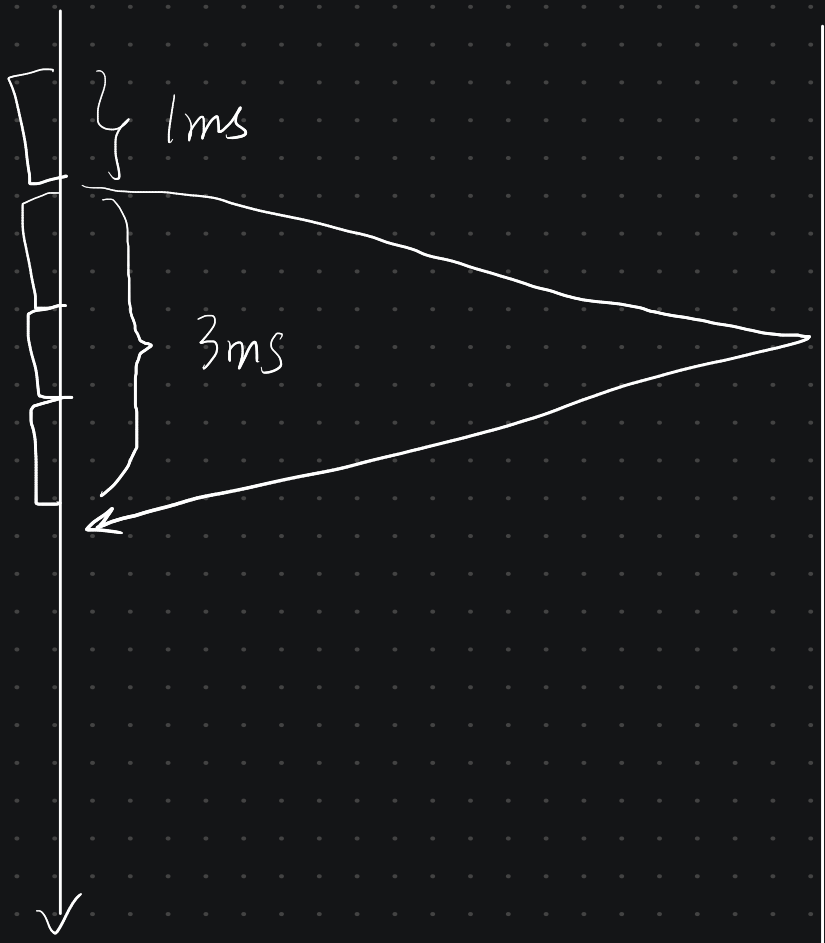
we can put 10 bits in 10 secs.

Capacity = $B \cdot T_p$

In stop & wait,
only 1 packet is present at a time.
The same packet moves forward till the end and then a new packet enters.

Instead, we could do pipelining,
↳ using sliding window protocol.

Sliding Window Protocol



when we're waiting for the 1st packet's ACK, we send more packets.

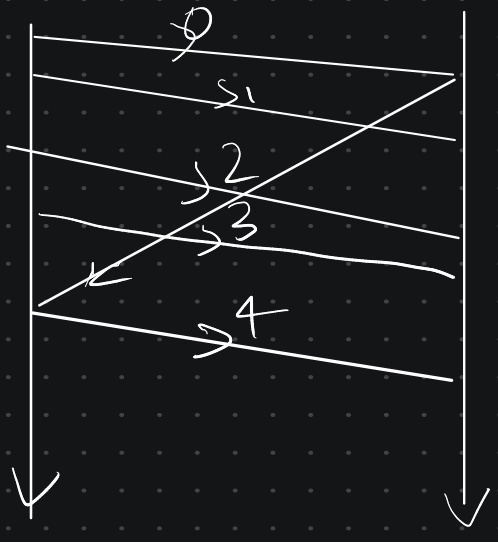
when packet is sent, it is put in a buffer till we get ACK. Then we discard it and put next packet in buffer.

i.e. How long to save packet?
→ till ACK is received (1 RTT).

---

on sending more packets, buffer should have them all.

3 2 1 0



Then once ACK of 0 is received, 0 will be discarded from buffer & 4 will be added. i.e. buffer/window will slide.

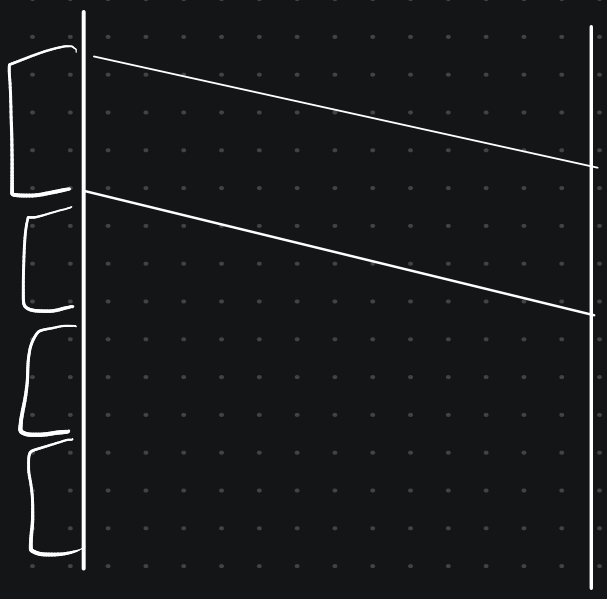
$T_t = 1$ ms, $T_p = 1.5$ ms. What should be the window size for 100% efficiency?

→

T_t time → 1 packet

$T_t + 2T_p$ → ?

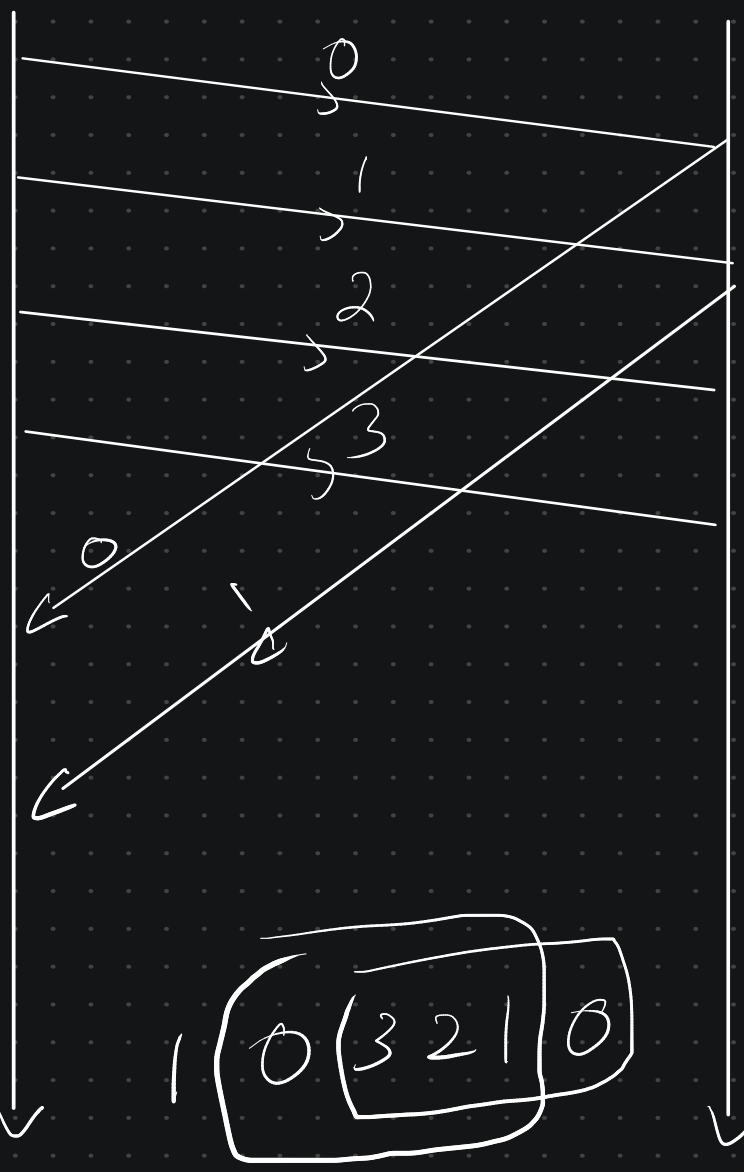
$(1 + 2a)$



window size (sender) = $1 + 2(1.5) = 4$

7 6 5 4 3 2 1 0

How many min. sequence #s do we need?



now once we get ACK 0 then 0 is no longer in the system. So we can reuse 0.

There shouldn't be overlap of sequence #s inside the window.

∴ min # of sequence #s = size of window.

E.g: If 16 sequence #s, how many bits?

→ 4 bits

E.g: 10 seq #s? 4 bits.

∴ With 4 bits we get 16 seq #s,
we want only 10.
So seq. #s can be greater than window size.

# of seq. #s $\geq$ sender window size

# of seq. #s $\geq (1+2a)$ for 100% efficiency.

# of bits in seq. # $= \lceil \log_2(1+2a) \rceil$

Q. $T_t = 1$ ms, $T_p = 49.5$ ms,

$|W_s|$ for 100% η?

↑ window size

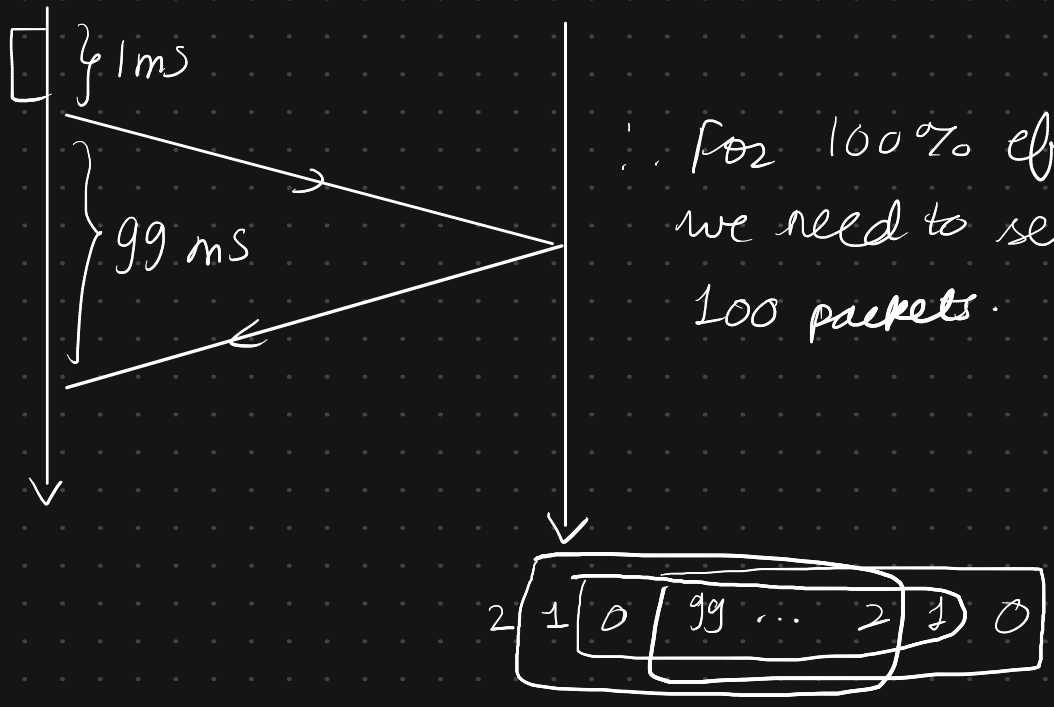
→ $1 + 2a = 1 + 2(49.5) = 100$

Q. Then what is the # of seq. #s reqd?

→ ≥ 100

Q. # of bits reqd?

→ $\lceil \log_2 100 \rceil = 7$ (128 seq #s possible using 7 bits)



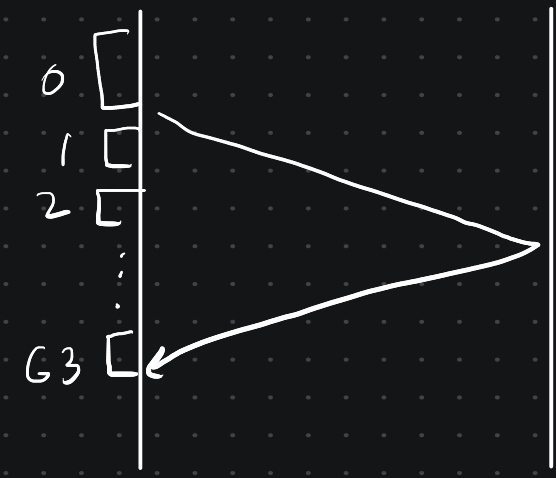
∴ For 100% eff. we need to send 100 packets.

we need seq #s 0-99 for sure. we could obv. have more.

Q. If # of bits reqd is 7, but we only have 6, then what is the efficiency?

→ Here we've decided on $W_S = 100$ for 100% eff.

If we only have 6 bits, we'll have only 64 seq. #s.



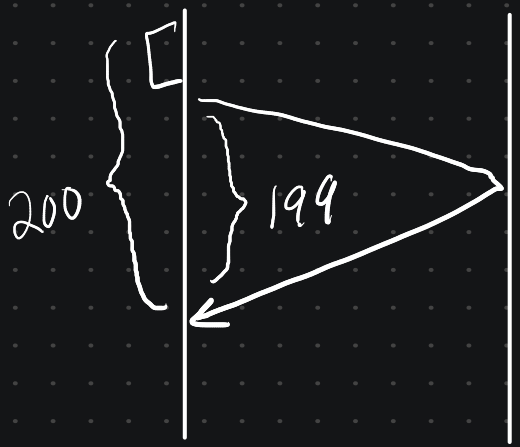
We have to wait for 0 ACK to come before we can reuse it.

$\therefore \eta = \frac{64}{100} = 64\%$

Q. $T_t = 1$ ms, $T_p = 99.5$ ms, $|W_s|$ for 100% eff.?

$\rightarrow 1 + 2a = 1 + 199 = 200$ # of seq. #s ≥ 200

$\therefore$ we need ≥ 8 bits $(2^8 = 256)$



Q. Say we only have 7 bits. $\eta = ?$

$\rightarrow 2^7 = 128$

$\eta = \frac{128}{200} = \frac{64}{100} = 64\%$

Q. $T_t = 1ms$, $T_p = 149.5\ ms$. $|W_s|$ for $\eta = 100\%$

→ $1 + 2a = 1 + 2(149.5) = 300$

Q. # of bits reqd. in the seq # field?

→ 9 bits

Q. If we only have 8 bits? $\eta = ?$

→ $\frac{256}{300} = 85.33\%$

Sender window size = $1+2a$ provided enough seq #s

If 'n' is the # of bits in seq. # field, then we have 2^n sequence #s.

Do we take window size $(1+2a)$ or 2^n? 2^n.

Sometimes $1+2a$ is just taken 2a. Just check op^n's.

$$W_S = \min(1+2a, 2^n)$$

All we've discussed till now is theoretical. Practically it is implemented as Go back N & SR.

SWP

Go-Back-N Selective Repeat

Go Back N:

Btw
$W > 1$.
If $W = 1$, it becomes stop & wait

i) Sender window size is N.

Q. $T_t = 1ms$, $T_p = 49.5ms$ Then what is the efficiency for go back 10?

→ $|W_s| = 10$ Ideally should be $1+2a = 100$

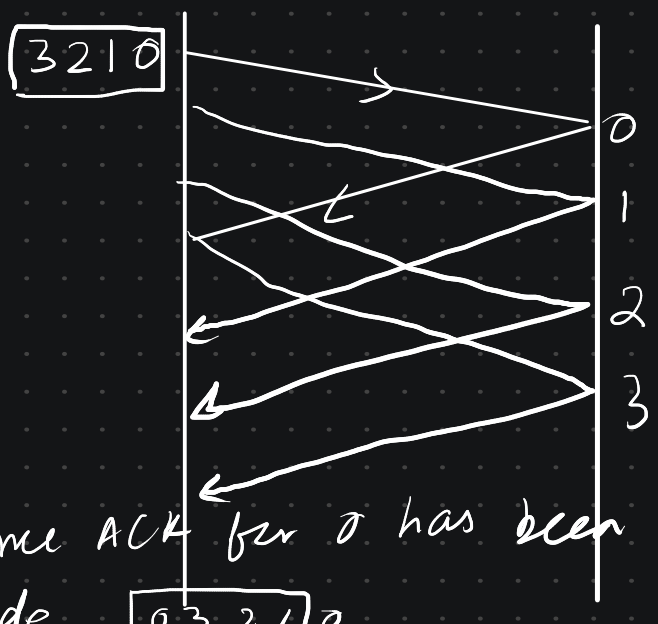
$\therefore \eta = 10\%$

Q. $T_t = 1ms$, $T_p = 99.5ms$, $\eta = ?$ for goback 100

→ $\frac{100}{200} = 50\%$

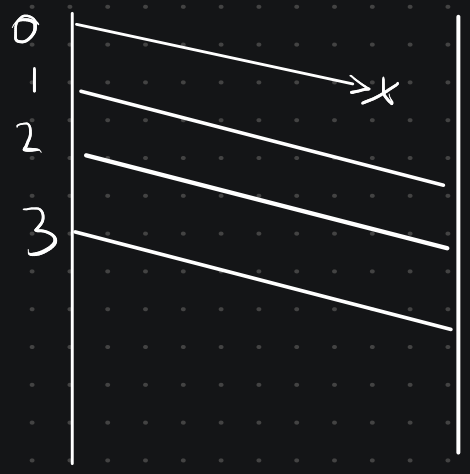
ii) Receiver window size is 1.

Say $|W_s| = 4$. i.e. go-back-4.



Receiver gets packet 0. Then 0 will be kept in buffer, processed, & discarded. Then buffer waits for 1. Then same for next.

Once ACK for 0 has been received, window will slide 0321 0



Say 0 is lost 1,2,3 aren't.
Receiver won't accept 1 as it is waiting for 0.

1, 2, 3 will be discarded at the receiver side

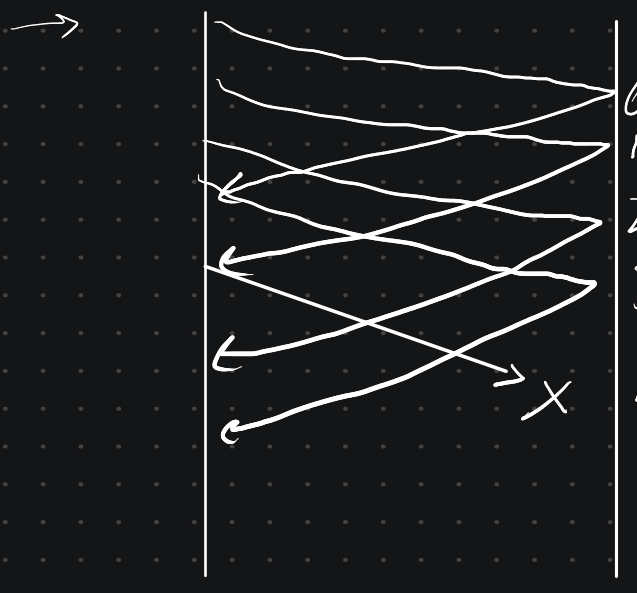
As at sender's, ACK isn't received, then 0, 1, 2, 3 will be retransmitted.

Go-back-4

Q. Say in GBN:

0 1 2 3 4 5 6 7.

Packet 4 is lost. What'll be retransmitted?



By the time we reach 4, 0 1 2 3 will be transmitted & acknowledged.

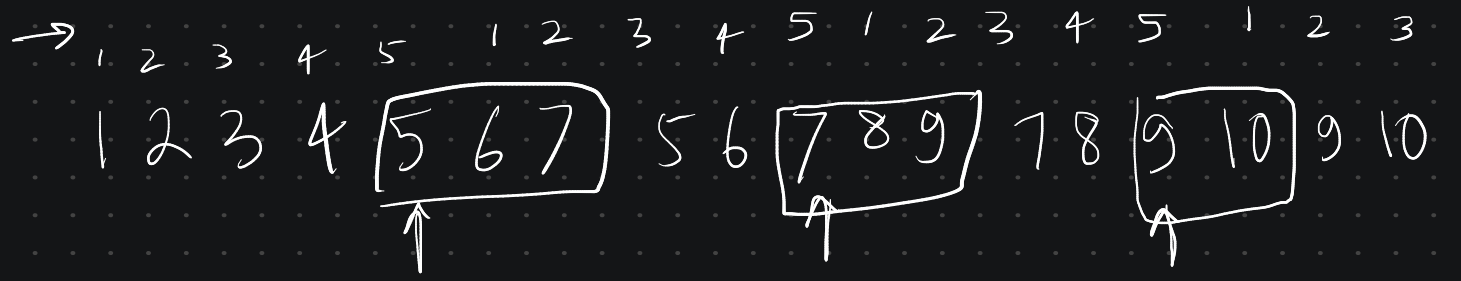
window will be (4 5 6 7)

They'll be discarded.

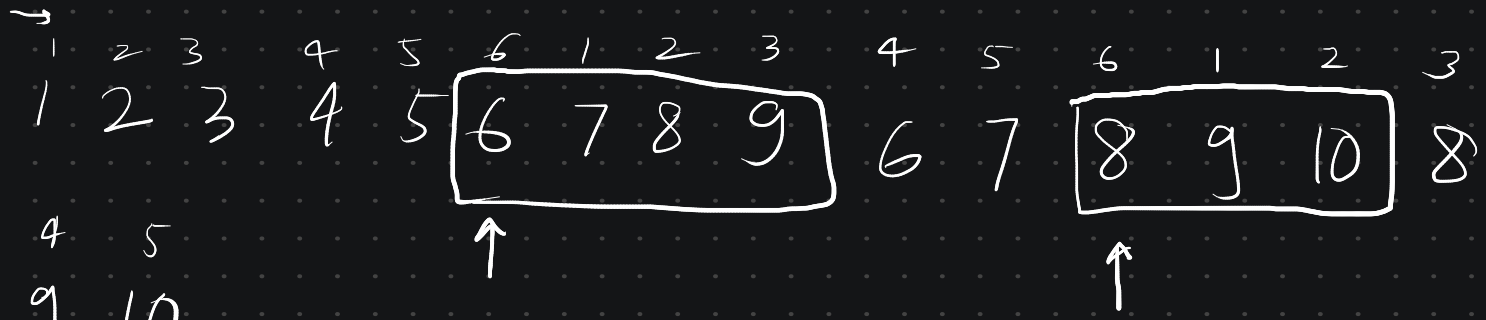
{4, 5, 6, 7} will be retransmitted.

Those in window are transmitted but not acknowledged.

Q. For G-B-3, if every 5th packet transmitted is lost and if we have to send 10 packets, then how many total transmissions are need?

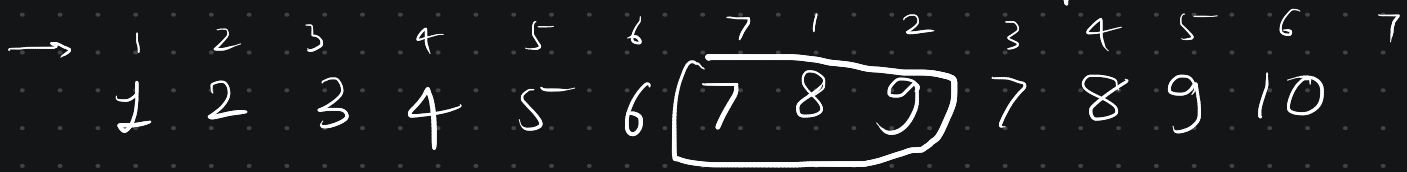


Q. GB4; every 6th packet is lost; Total 10 packets; Total # of transmissions?



= 17

Q. GB3; every 7th lost; Total 10 packets;



= 13

GBN uses cumulative ACK. ACK ← Independent / Cumulative

i.e. If we get ACK 10, it means packets 0 to 9 are received as well.

Receiver window size is 1. So we'll receive in-order packets only.

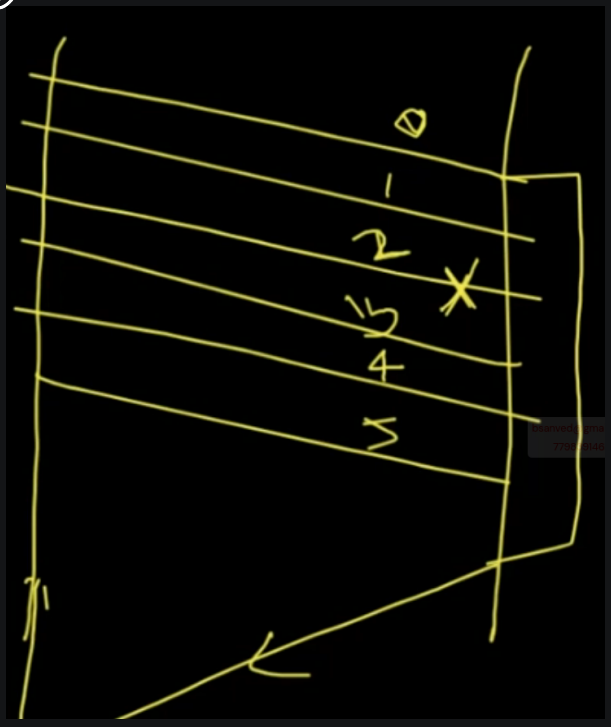
We don't need ACK for every packet as out-of-order packets are not allowed.

Also there'll be an ACK timer on the sender's side.

Say 1 2 3 _ 5 6
(5 ×, 6 ×)

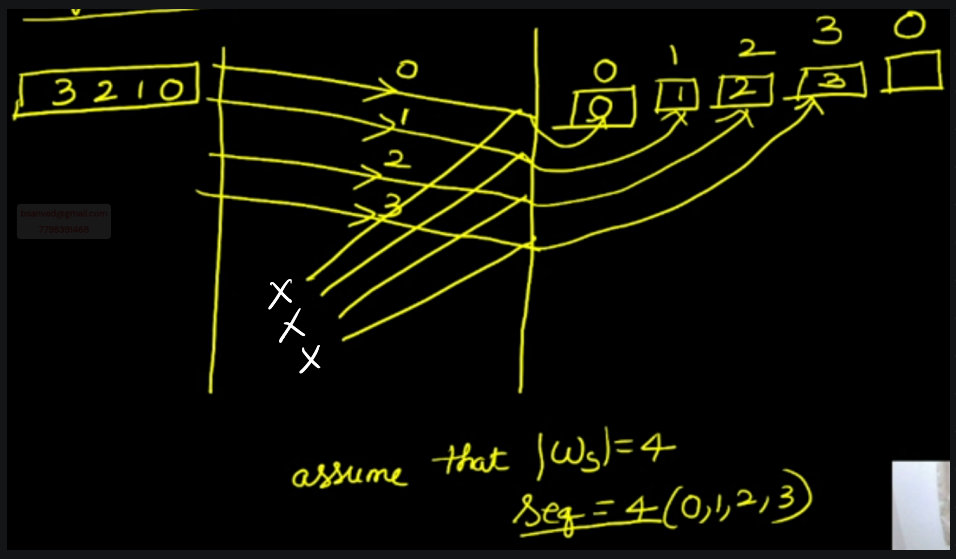
If 4 is lost, 5, 6, ... will be discarded.
So after a while 4 will be retransmitted due to ACK timer on sender's.

e.g.



If 2 is lost.
ACK # would be 2.
i.e. Those before 2 are received.
2 is lost.
3, 4, 5, ... are discarded.

Seq. #s needed in go-back-N:



Say receiver gets 0, 1, 2, 3, sends back ACK, but they are lost.

Now receiver will wait for new packet whose seq # will also be 0. But sender will now send old 0. So the old packet will be collected again.

↳ Duplicate packets

To solve the duplicate packet problem, we need more seq. #s.

If we had one more seq. #, say 4, the receiver wouldn't accept the old 0 as it'd be waiting for 4.

So if sender window size is 4, we need 5 seq. #s.

$$4 + 1 = 5$$

$$|W_s| \quad |W_R|$$

So we get a rule,

Rule: In any implementation of SWP,

$$\text{seq. \#} \geq W_S + W_R$$

$$\uparrow \quad \uparrow$$

$$N \quad 1 \quad \leftarrow \text{for GBN}$$

Q. How many seq. #s in go back 100? → ≥ 101

Q. If seq. #s available are N, then what is sender & receiver window size in GBN?

→ $W_S = (N-1)$, $W_R = 1$.

Q. If k bits are available in seq. # field, then $W_S, W_R = ?$

→ k bits ⇒ 2^k seq #s.

⇒ $W_S = (2^k - 1)$, $W_R = 1$

Selective Repeat Protocol:

Sender window size = receiver window size.

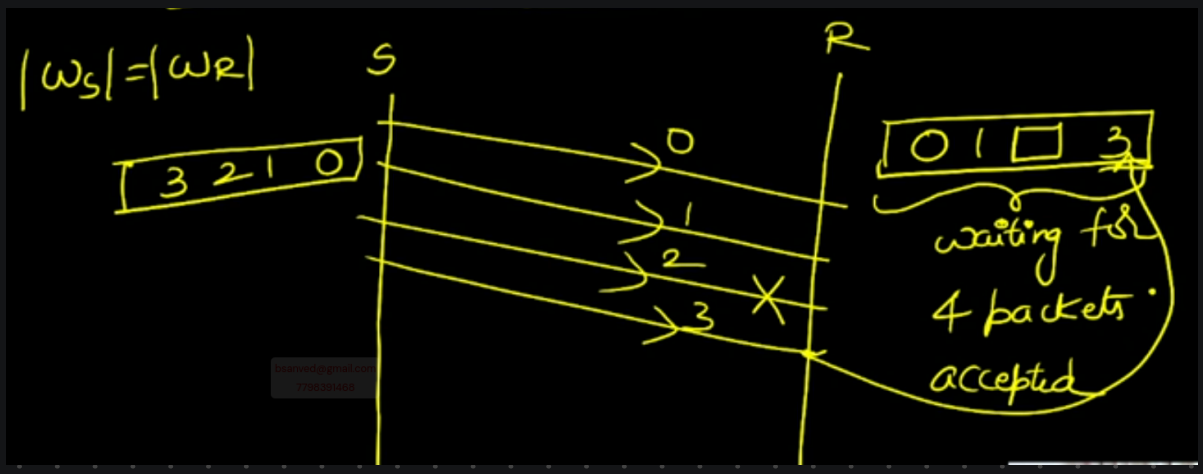
$W_S = W_R$

Q. $T_t = 1\,ms$, $T_p = 49.5\,ms$, $W_S = 50$, $\eta = ?$

→ $1 + 2a = 100$

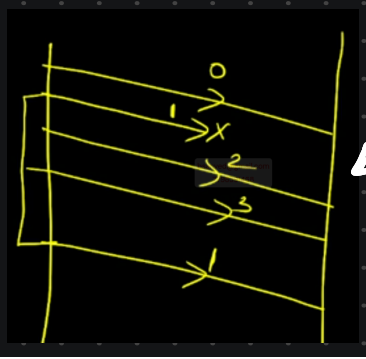
$\eta = \frac{50}{100} = 50\%$

$|W_S| = |W_R|$ As sender sends 4 packets, receiver is waiting for 4.



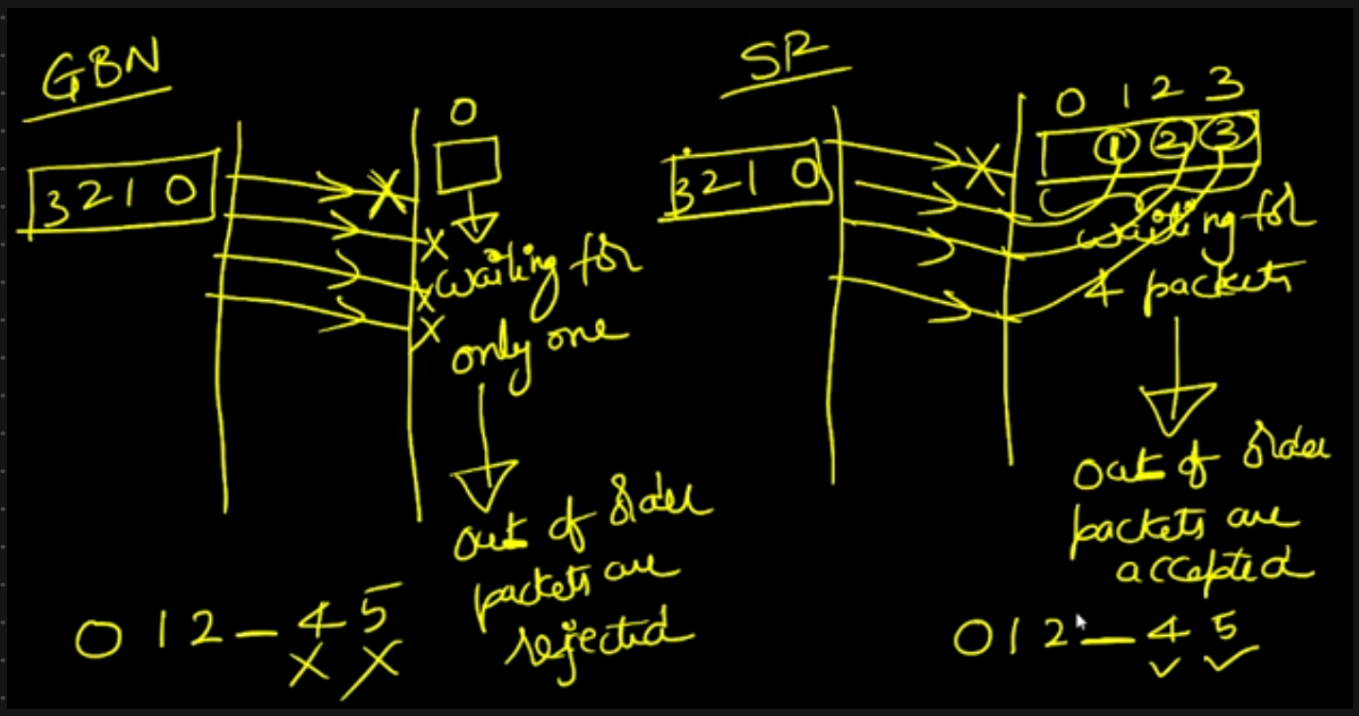
Say a packet '2' is lost. But receiver is waiting for 3 packets. So if 2 is lost, 3 is not discarded.

Out-of-order packets are accepted.



unlike in GBN, entire window will not be retransmitted.

Only the lost packet, i.e. 1 here will be retransmitted.



Q. In SR, $W_S = 3$, 10 packets to be sent. Every 5th packet is lost. # of total packets transmitted?

→ 1 2 3 4 5 5 6 7 8 9 9 10 — 12

(1 2 3 4 5 — 1st retransmission of 5; 6 7 8 9 — 5th retransmission of 9)

η → for GBN and SR → high

retransmissions → for SR and S&W → low

Q. SR: 10 packets, every 4th lost.

→ 1 2 3 4 4 5 6 7 7 8 9 10 10 — 13

(4 — 1st retransmission of 4; 7 — 2nd retransmission of 7; 10 — 3rd retransmission of 10)

ACKs in SR:

In GBN,
say we send 0 1 2 3 4. If we receive ACK for 4, i.e. ACK 5, it means 1, 2, 3 are received properly as well. (Else 4 wouldn't have been accepted)
So cumulative ACK will work.
1 ACK can acknowledge many packets.

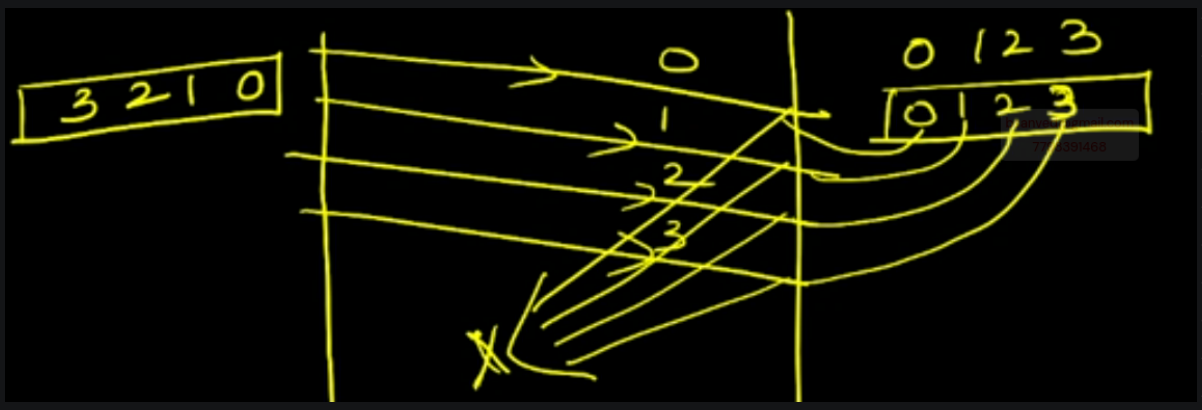
In SR,
if we get ack. for 4, i.e. ACK #5, it doesn't mean anything for 0, 1, 2, 3.
One ACK can't do many packets.
So cumulative is NOT suitable for SR.

SR uses independent ACK.
↳ has high traffic

SR also uses negative ACKs. (Only SR does)
If some packet, say 2 is corrupted, then NAK 2 will be sent.

+ve → packet received. (Bit → 0)
-ve → packet corrupted (Bit → 1)
} will have an extra bit.

Seq. #s in SR:



If all ACKs are lost, old 3210 will be retransmitted. And they're received as new.

↳ Duplicate packet problem.

If $|W_S| = 4$, then we need 4 extra seq. #s.
Receiver would wait for 4, 5, 6, 7.

$$\#\text{ of seq. \#s} \geq |W_S| + |W_R|$$

Say there's a new protocol. $|W_S| = 100$, $|W_R| = 50$.
We'd need 150 seq. #s.

In SR, $|W_S| = N$, $|W_R| = N$

$$\#\text{ of seq. \#s} \geq 2N$$

If # of seq. #s $= N$,
then $|W_S| = |W_R| = N/2$.

If k bits in seq. # field, then
$|W_S| = |W_R| = 2^{k-1}$

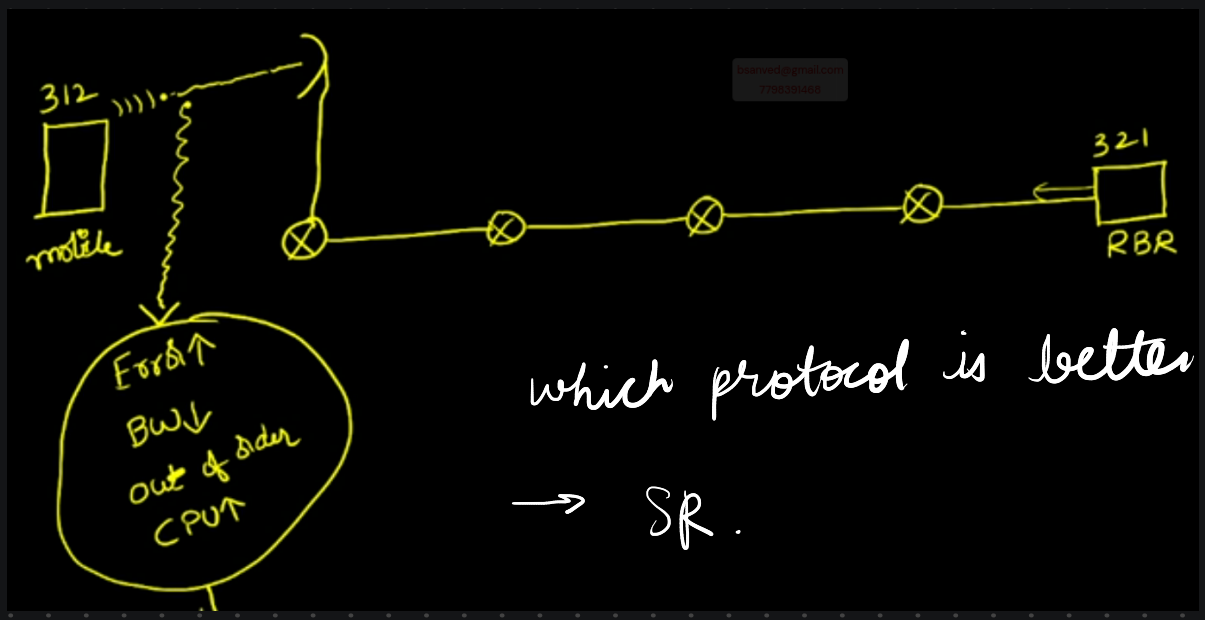
BTW, $|W_S| = |W_R|$ gives the max. η.

	Stop and wait	go-back-N	selective repeat
η	$\left(\frac{1}{1+2a}\right)$	$\left(\frac{N}{1+2a}\right)$	$\frac{W_S}{1+2a}$
Buffers ($W_S + W_R$)	$1+1$	$N+1$	$W_S + W_R$ $= W_S + W_S$
Seq. #s	$1+1$	$N+1$	$W_S + W_R$ $= W_S + W_S$
Retransmi's	1	N	1
BW	low	high (due to RTs)	moderate
CPU	low	moderate	high

In SR, out-of-order packets are accepted.
So **sorting** is reqd. at receiver's.
Also to send lost packet, **searching** is reqd. at sender's. ∴ CPU reqs. are high

Implementation	easy	moderate	complex

Q.



which protocol is better?

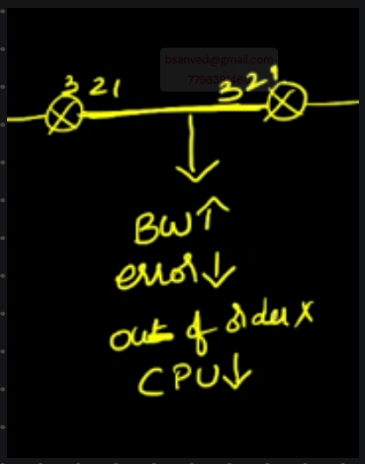
→ SR.

SR is good for end-to-end.

As the error rate is high, we won't use GBN.

Also as we get out-of-order, much discarding would take place.

Q.



In physical connects, which protocol?

→ go-back-N.

↳ good for hop-to-hop (1 link)

TCP uses SR.

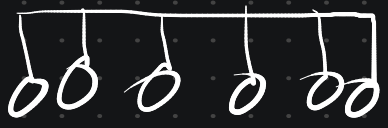
DLL protocols like HDLC use go-back-N.

Stop and wait isn't used practically used as η is too less.

Access control methods

links

- Broadcast



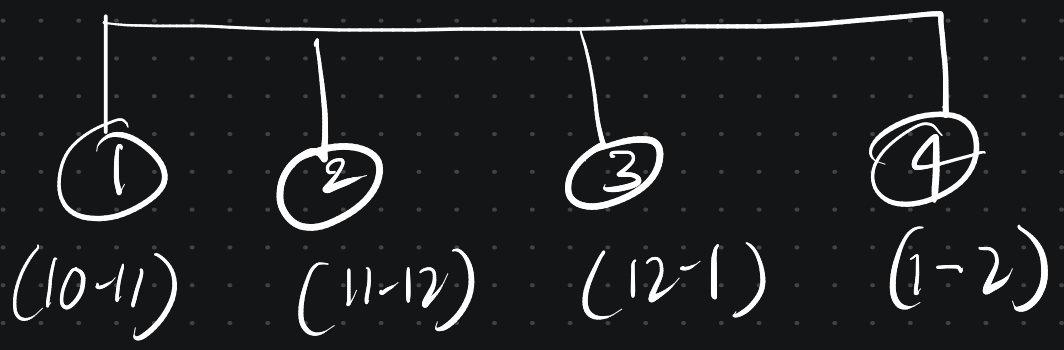
Problem:
when 2 stations send data at same time,
⇒ collisions

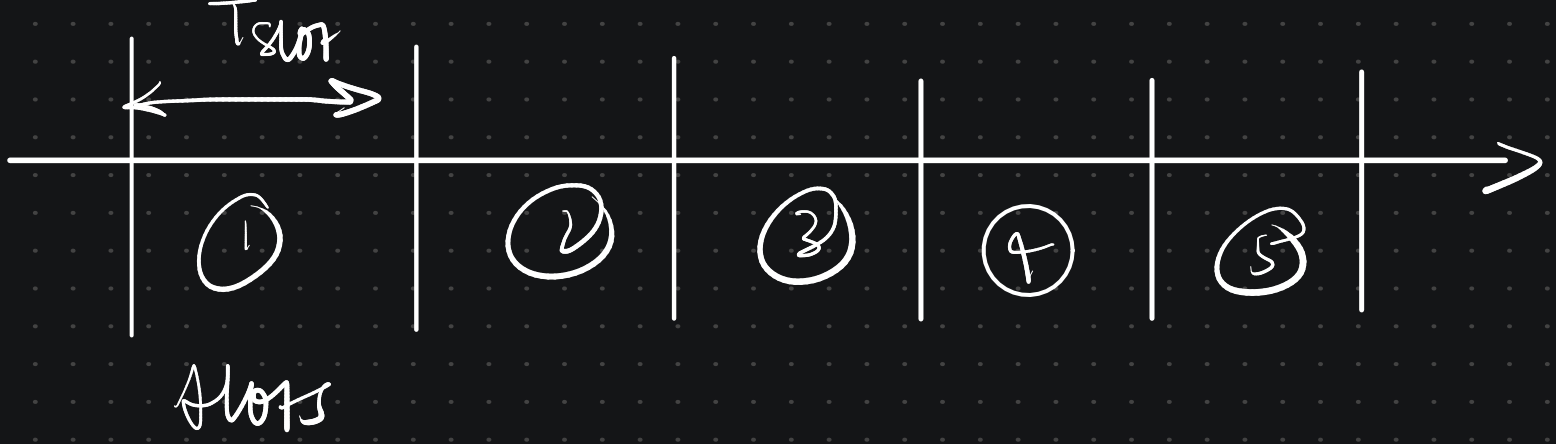
- point-to-point

full duplex
2 sta^s can transmit together

access control:
allowing only 1 station at a time.

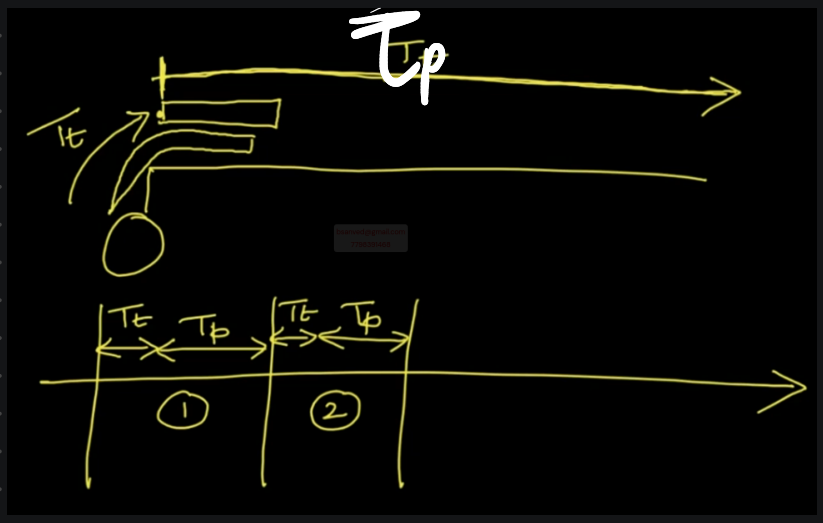
TDM : Time Division Multiplexing





If station is given time slot, it must (in that time) put packet on channel & last bit must go out.
Each slot should have time of

$$T_t + T_p$$



1 cycle

assumption: all hosts have same T_t (i.e. same packet size)

(T_p is same for entire channel anyways)

$$\eta = \frac{\text{useful time}}{\text{cycle time}} = \frac{T_t}{T_t + T_p} = \frac{1}{(1+a)}$$

Q. $T_t = 1ms$, $T_p = 1ms$, TDM. $\eta = ?$

→ $\eta = \frac{1}{1+\frac{1}{1}} = \frac{1}{2} = 50\%$

Q. BW = 4 Mbps, Throughput = ?

→ Throughput = effective BW

$= \eta * B$

$= \frac{1}{2} \times 4 Mbps$

$= 2 Mbps.$

BTW we're dealing w/ access control.

ACK is considered a packet, same as data.

8.

one station requires 2Kbps, then max no of stations that can be connected to above n/w?

$T_t = 1ms$, $T_p = 1ms$, TDM $\eta = 50\%$

BW = 4 mbps → Throughput = ___ ?

$Th = BW \times \eta = 4 * 0.5 = 2$ real mbps

→ Throughput / eff. BW = 2×10^6 bps

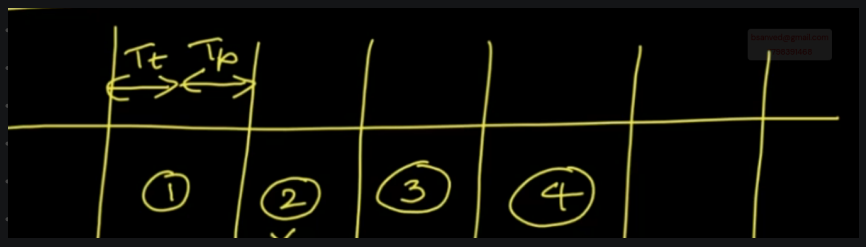
Say N stations.

$$N \times \underbrace{2 \times 10^3 \text{ bps}}_{\text{requirement of 1}} = \underbrace{2 \times 10^6 \text{ bps}}_{\text{total available}}$$

$$N = 1000$$

Disadvantages of TDM:

- All stations are given particular time slots.

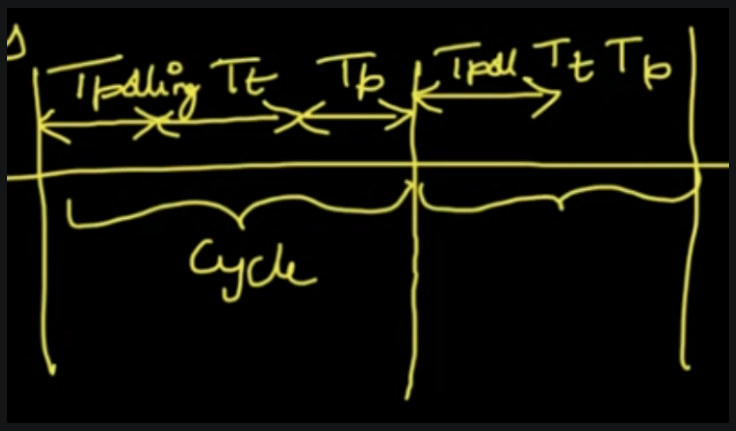


Say sta^ 1 transmits data.
Then 2's turn.

But say 2 does not have data to transmit.
That slot will be wasted altogether.

Better way would be polling.

Polling: Polling algo. will be used to determine who sends data next.



$$\eta = \frac{\text{useful time}}{\text{cycle time}}$$

$$= \frac{T_t}{T_t + T_p + T_{poll}}$$

Q. If $T_p = T_t = T_{poll} = 1ms$, $\eta = ?$

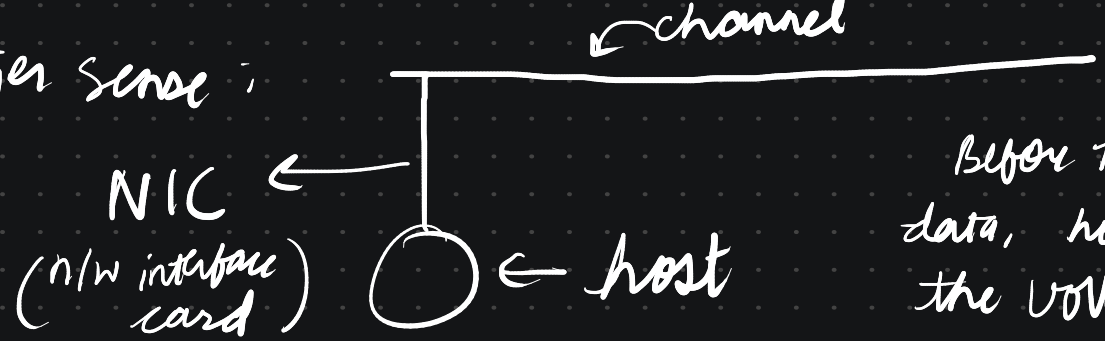
$$\rightarrow \frac{1}{1+1+1} = 33.33\%$$

always 1 ques^n in GATE

CSMA/CD :

Carrier Sense Multiple Access / Collision Detection

Carrier Sense :



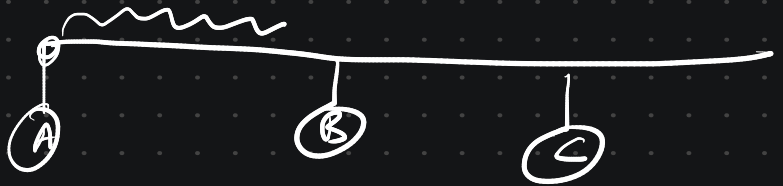
Before transmitting data, host senses the voltage.

If voltage = 0, i.e. no data, then it transmits.

Multiple Access : multiple stations connected to the channel.

Collision Detect :

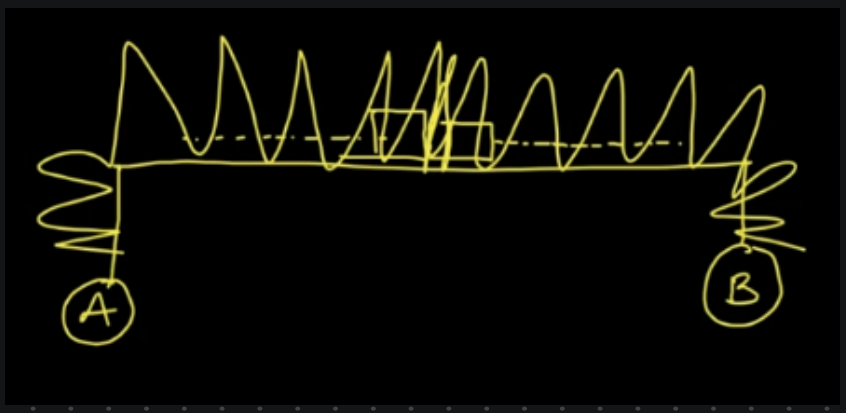
Say no data on channel. A starts.



how to detect that data is lost?

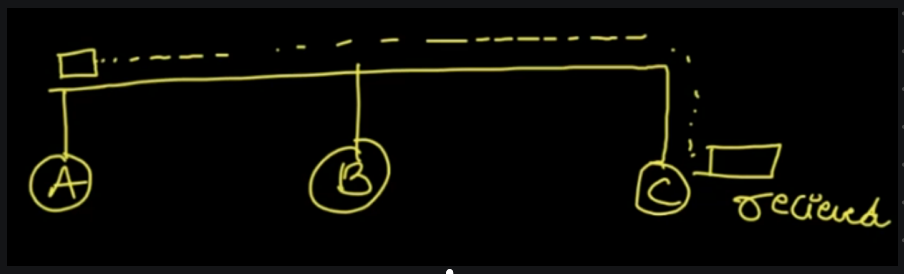
A sends to a certain point.
B doesn't see it. B transmits.
Then collision happens.

BTW : collision signal is a garbage value



How will A & B know that their own data is lost?

A collision signal won't always mean that data is lost:

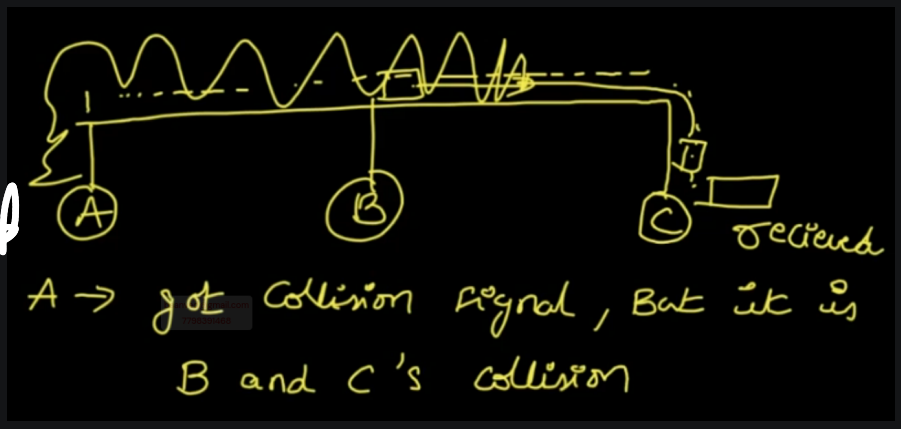


say A sends to C. succesfully.

say B and C send something & collision occurs.

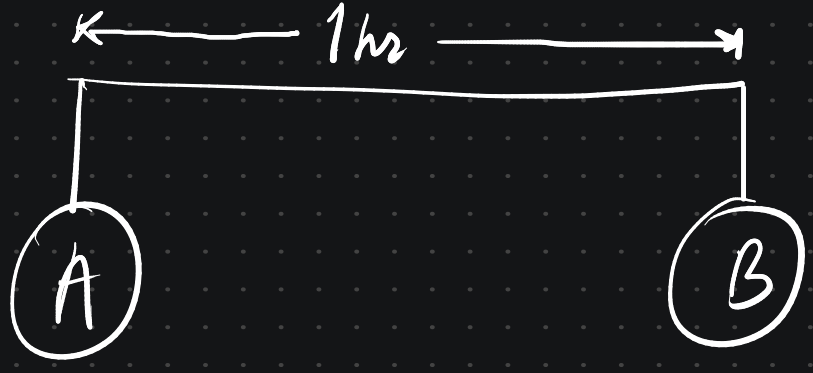
A's data is delivered

But A gets irrelevant collision signal.



If A is transmitting and gets collision signal while transmission, then A's packet is lost.

If A is still transmitting on collision, A's data is definitely lost.



Say T_p = 1 hr

Say at t = 10:00 a.m., A, B start transmitting then at t = 10:30 a.m., collision would occur.

At what time will A, B see the collision?

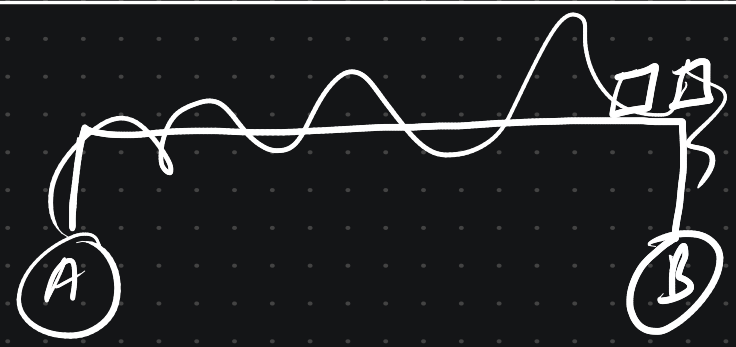
11:00 a.m.

So in this case, how long did it take from beginning till collision signal?

01:00 hr.

---

worst case:



A starts at 10:00 a.m.

At 10:59:59:59, A's bit will be almost at B.

But say B starts then. Then collision occurs

A will see collision at ≈ 12:00:00 pm

Total Time = 02:00 hrs.

Worst case : collision signal comes after $2 * T_p$ time.

In order to detect collision, it should still be transmitting.

So $T_f \geq 2 * T_p$ to see the collision.

So small packets cannot use CSMA/CD.

Ethernet (LAN) uses CSMA/CD.

min packet length:

$$T_p \geq 2 \cdot T_p$$

$$\frac{L}{B} \geq 2 \cdot T_p$$

$$\rightarrow \boxed{L \geq 2 \cdot T_p \cdot B}$$

Q. $T_p = 1\,ms$, $B = 4\,Mbps$, L_{min} in CSMA/CD = ?

$$\rightarrow L \geq 2 \cdot T_p \cdot B$$

$$\geq 2 \cdot (1 \times 10^{-3}) \cdot (4 \times 10^6)$$

$$\geq 8 \times 10^3 \text{ bits}$$

$$\geq 10^3 \text{ B}$$

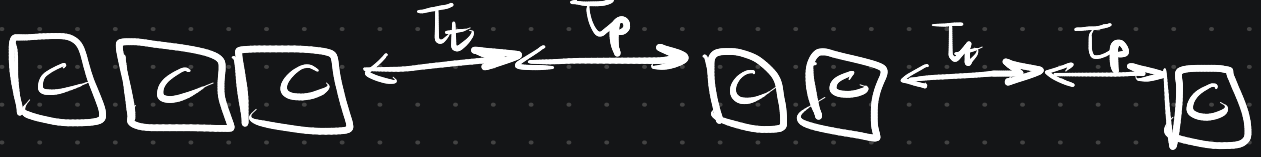
∴ In CSMA/CD, we've to send atleast 10^3 B.

Say we only have 900B. Then we'll add 100 dummy Bytes. This is **Padding**.

Efficiency of CSMA/CD:

difficult to get direct formula → Because 2 stations may try to send data. collision.
Then retransmissions, then collisions.

we don't know how many collisions would take place.

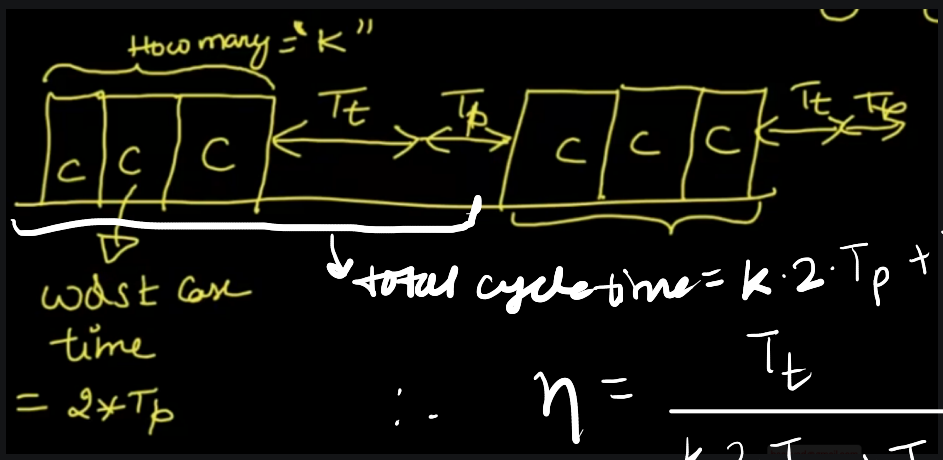


Any # of collisions can take place.

After some collisions, the stations would be able to transmit properly.

Worst case time for each collision = $2 * T_p$.

Say k collisions occur.

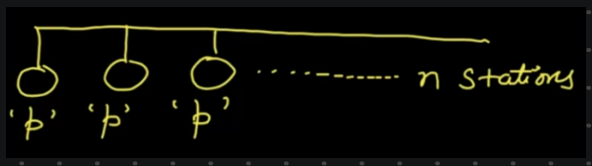


we can't get k directly.

$$\therefore \eta = \frac{T_t}{k \cdot 2 \cdot T_p + T_t + T_p}$$

Probabilistic Analysis to find K:

1) Let there be n stations and p be the prob. that a station wants to send data.



2) Prob. of successful transmission:
(only 1 station sends, others don't)

p for sending, (1-p) otherwise

$$P_{success} = {}^{n}C_1 \, p^1 \, (1-p)^{n-1}$$

To get maxima:

$$\frac{d P_{success}}{dp} = 0$$

we get $p = \frac{1}{n}$.

∴ At $p = \frac{1}{n}$, we get maximum value of $P_{success}$

$$P_{max} = n \cdot \frac{1}{n} * \left(1 - \frac{1}{n}\right)^{n-1}$$

$$P_{max} = \left(1 - \frac{1}{n}\right)^{n-1}$$

Imagine there are a large # of stations.

$$\lim_{n\to\infty} P_{max} = \lim_{n\to\infty} \left(1-\frac{1}{n}\right)^{n-1} = \frac{1}{e}$$

∴ max. value of success is $\frac{1}{e}$.

If p is prob. of getting head, how many times should we toss before getting 1st head?

→ $\frac{1}{p}$ using Poisson's dist.

∴ If $\frac{1}{e}$ is the prob. of a successful transmission, we should try e times to get the 1st success.

i.e. we fail the 1st e times.

i.e. e collisions $\quad K = e$

# of collisions = e

$$\therefore \quad \eta = \frac{1}{e * 2 * T_p + T_t + T_p}$$

$$\boxed{\eta \approx \frac{1}{1 + 6.44a}}$$

← efficiency of CSMA | CD.

$$= \frac{1}{1 + 6.4 \frac{d}{V} * \frac{B}{L}}$$

← use this to get relations like if $d \uparrow \eta \downarrow$.

$d \uparrow \eta \downarrow \Rightarrow$ CSMA is not good for WANs, better for LANs.

$L \uparrow \eta \uparrow \Rightarrow$ better for longer packets.

$B \uparrow \eta \downarrow \Rightarrow$ better for low BW.

In CSMA/CD →
there's no ACK
∴ collⁿ detecⁿ is needed.

Exponential Back-off Algorithm:

Say you call your GF, and she's calling you.
Both seem engaged on a call for the other.
Both try again. Still same.
How do you decide when to call/stop?
Both might stop/call at the same time.

Soln:
You tell your GF,
choose a random # from 0-10.

Say you choose 5. She chooses 2.
You'll wait for 5 mins, she for 2.
So GF will call after 2 mins.

There should be a waiting time for the stations.

Say both are sending packet #1,

on collision, both get collision # n = 1
Both will generate # from (0 to $2^n - 1$)

Here $2^{1-1} = 2^0 = 1$
from 0 to 1.

cases:

A	B	
0	0	→ collision (both send at once)
0	1	→ A will send immediately, B will send after $1 * T_{slot}$
1	0	→ B will send at once, A will wait for 1 T_{slot}.
1	1	→ collision (both send after $1 * T_{slot}$)

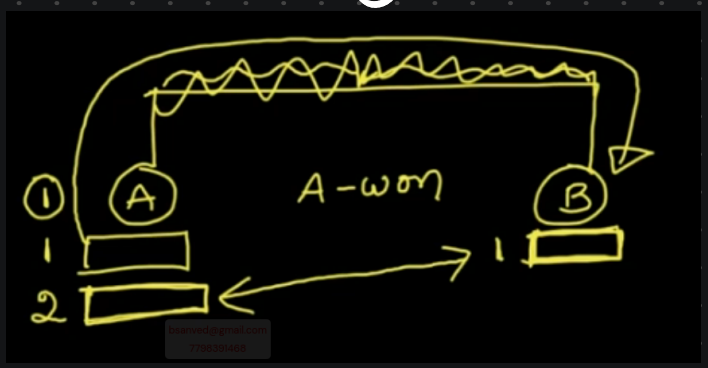
here T_{slot} is determined exptally.

$P(A) = P(B) = 1/4$ $P(\text{colli}^n) = 1/2$

BTW this is also called binary back-off algo.
(for 2 stations only).

Say A has won.
Then A will transmit successfully.

A is now ready with the 2nd, while B is ready with the 1st packet.

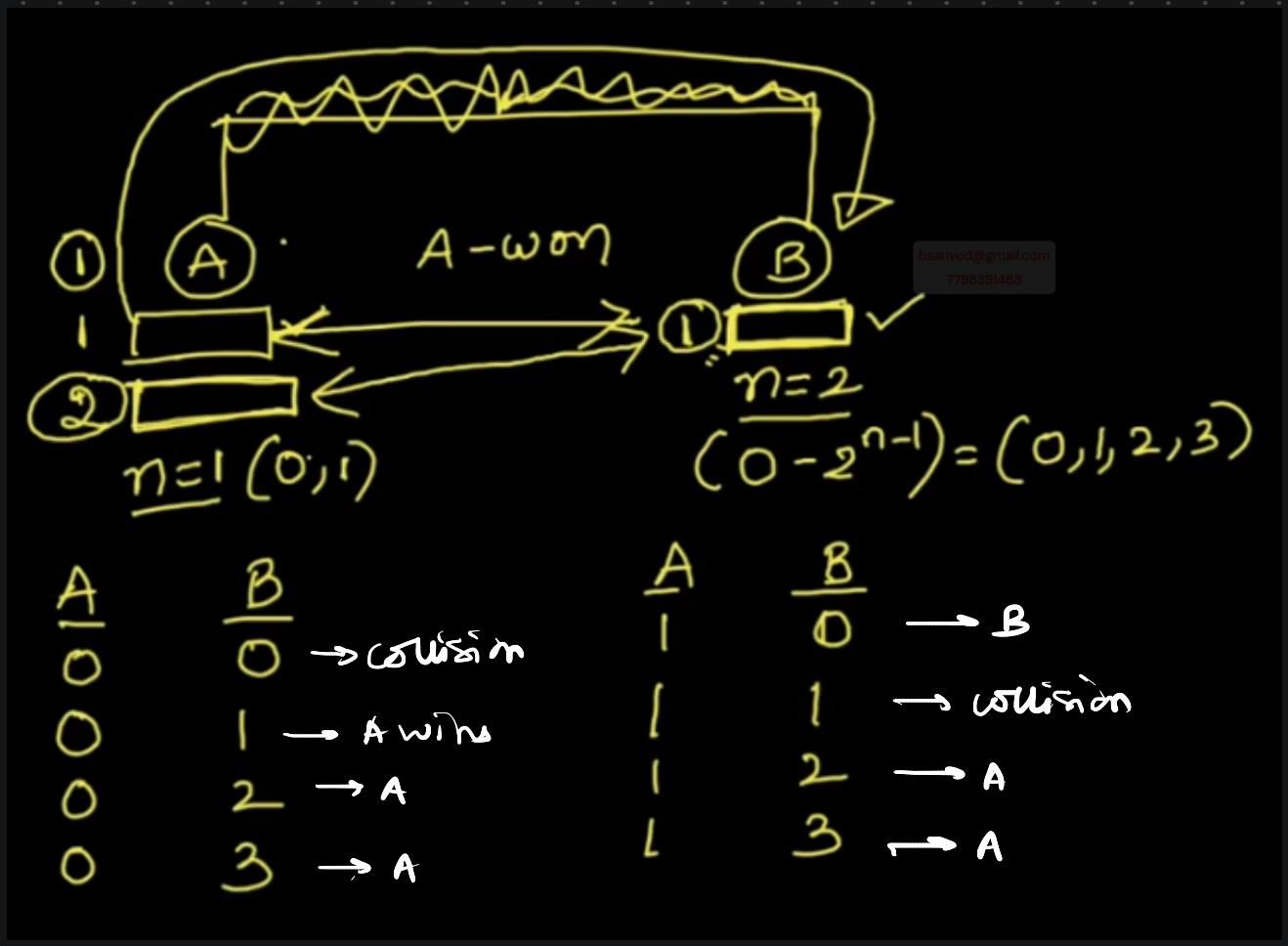


Say there's collision betn 2nd packet of A and 1st packet of B.

Here collision # for 2nd packet of A = 1 (colliding for the 1st time)

collision # for 1st packet of B = 2 (2nd time)

So for A : 0 to 2^{1-1} , B : 0 to 2^{2-1}



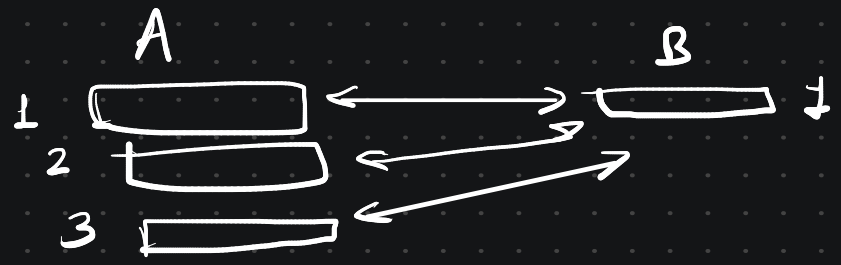
$\therefore P(A) = 5/8$ (given that A won in the 1st collision.)

$P(B) = 1/8$ (lost in the 1st)

$P(\text{collision}) = 2/8$.

Say (1, 2) happens. A wins.

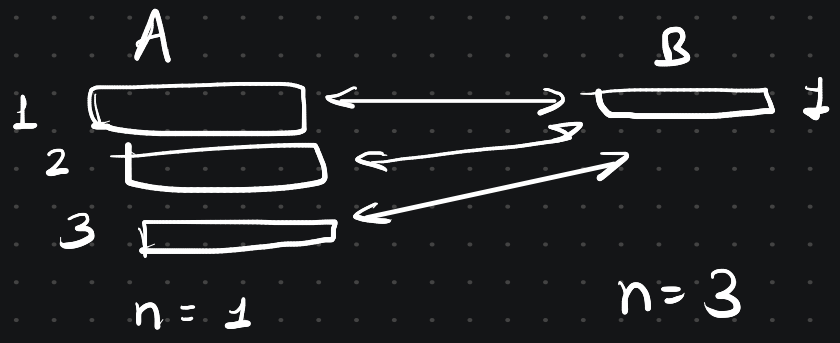
Now A → 3rd packet, B → 1st packet.



Say there's a collision again:

For A's packet #3, collision # = 1.

For B's packet #1, collision # 3.



(0 to 1) 2 choices (0 to 7) 8 choices 16 pairs

(0,0) & (1,1) → collisions. $P(C) = 2/16$

(0,1) → B wins. $P(B) = \frac{1}{16}$

Rest A. $P(A) = \frac{13}{16}$

A	B	
0	0	C
0	1	
0	2	
0	3	
0	4	
0	5	
0	6	
0	7	

A	B	
1	0	B
1	1	C
1	2	
1	3	
1	4	
1	5	
1	6	
1	7	

$P(A) = 13/16$

$P(B) = 1/16$

$P(C) = 2/16$

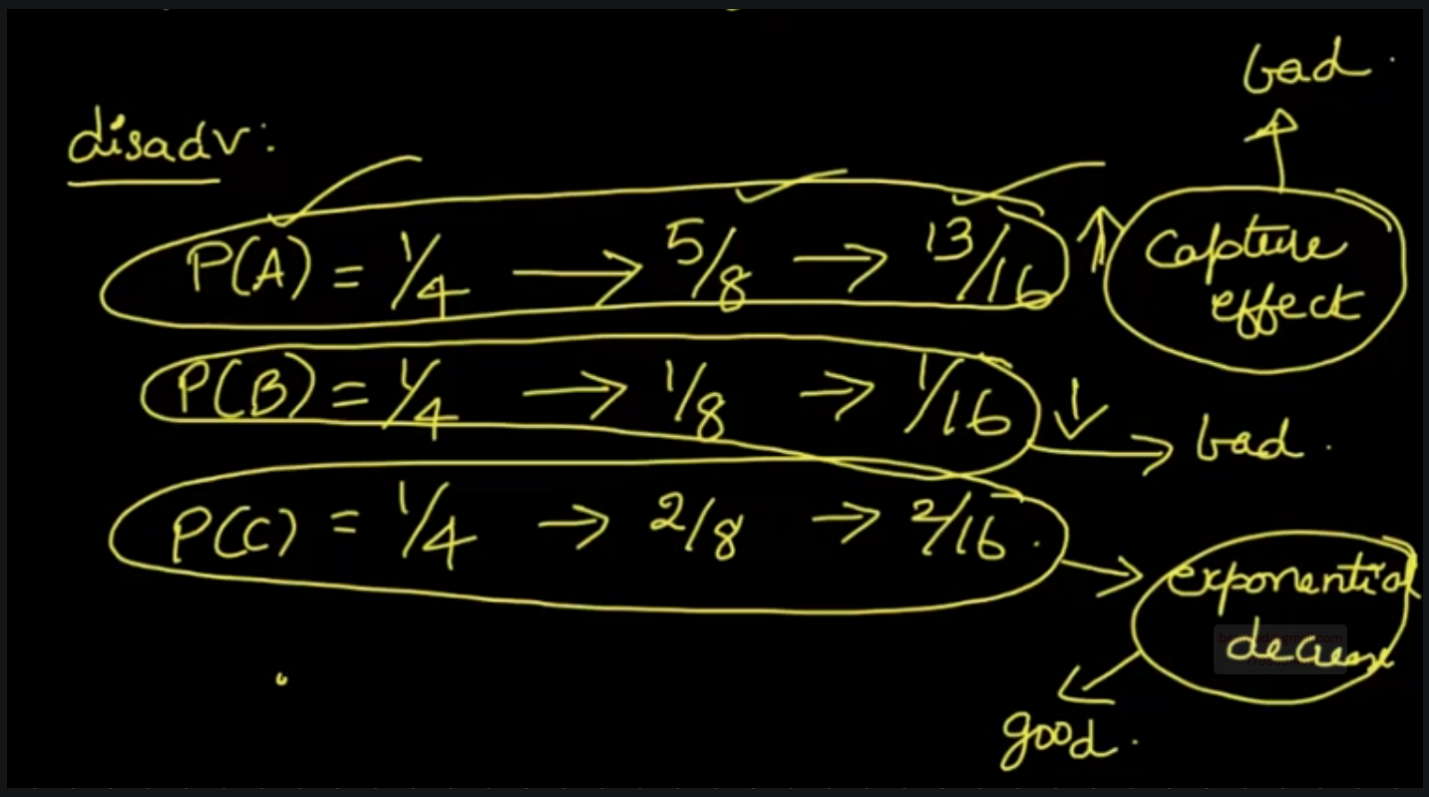
Disadvantage: Initially, $P(A) = P(B) = 1/4$.
Then if A wins, $P(A)$ ↑↑ to $5/8$
then next to $13/16$.

$P(B)$ decreases $1/4 \rightarrow 1/8 \rightarrow 1/16$

$P(\text{collision})$ $1/4 \rightarrow 2/8 \rightarrow 2/16$.

Whoever wins once, wins again & again
This is called "capture effect".
(A captures the link again and again)

Prob(collision) decreases exponentially.
This is good.



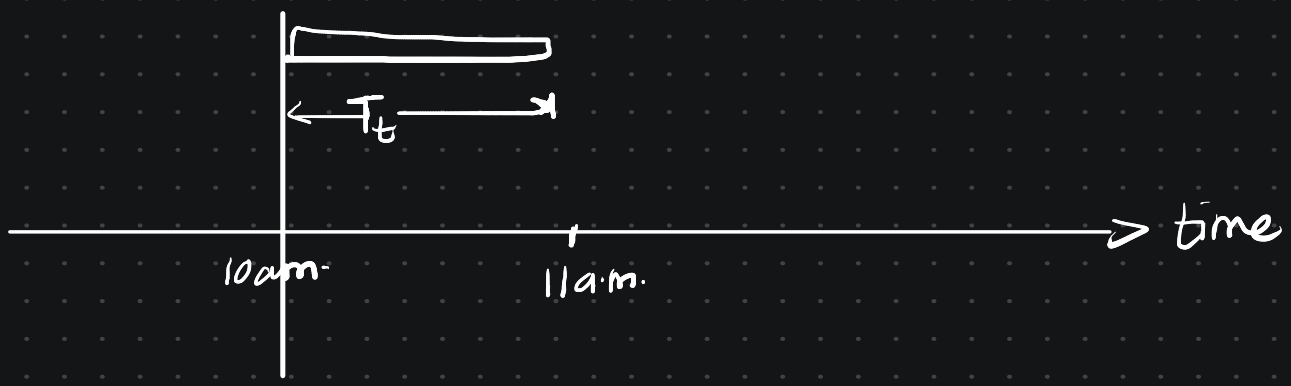
Also, the algo is only used betⁿ 2 stations.
(binary)

Pure ALOHA :

- → Any station can send data anytime.
- → Collisions are possible.
- → ACKs are present (so no colli^ detec^ needed)
- → Retransmit lost packets.

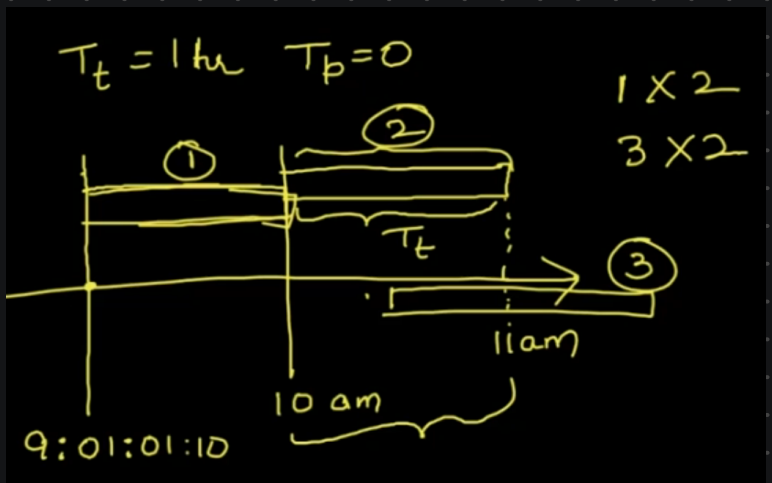
ALOHA is used in wireless transmi^.

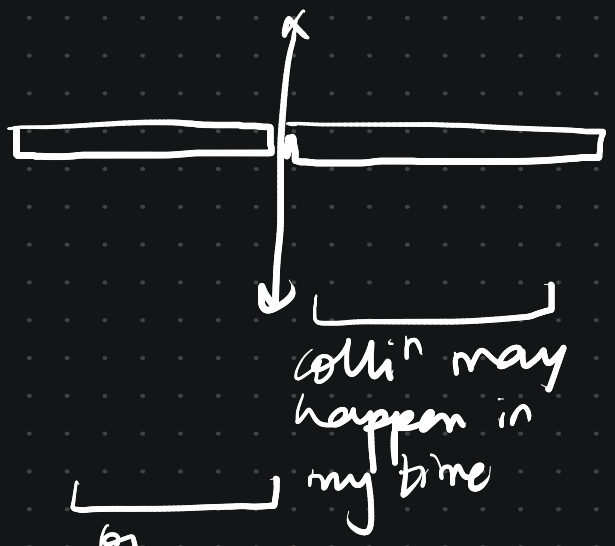
Say $T_t = 1\,hr$, $T_p = 0$.



If there's a collision, it'll occur bet^ 10 a.m to 11 a.m.

say someone has started a packet somewhere at 9:01.





Vulnerable Time $\simeq 2 * T_t$

↳ For Pure ALOHA

For pure ALOHA,

$$\eta = G_1 * e^{-2G_1}$$

G_1 = # of stations who want to transmit in the T_t slot.

For η_{max}, $\frac{d\eta}{dG} = 0$

$\Rightarrow G_1 = 1/2$ $\quad \eta_{max} = 18.4\%$

G is # of stn's that want to transmit data in 1 T_t.

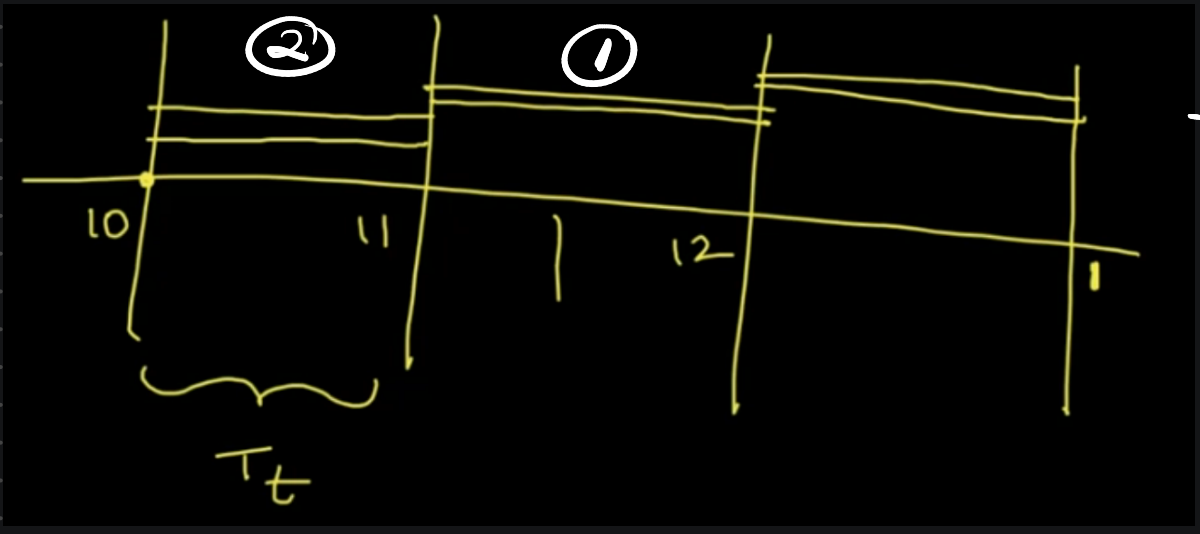
½ station means in $2T_t$, 1 station transmits.

BTW ALOHA isn't too imp. for GATE. Remember the main stuff only.

Slotted ALOHA:

Like in TDM, time is divided into slots.

Just assume $T_p = 0$.



Stations have to start at beginning of time slots, not in middle.

So packet #1 won't collide with 2.

Only collisions could occur if 2 stations transmit exactly at the beginning of time slot.

$$\text{Vulnerable Time} = T_t$$

$$\Rightarrow \eta_{\substack{\text{SLOTTED} \\ \text{ALOHA}}} = G \cdot e^{-G}$$

$$\frac{d\eta}{dG} = 0$$

$$G = 1$$

1 station wants to transmit in $1T_t$.

$$\eta_{max} = 36.8\,\%$$

Error Handling

- Error Detection
  - sending the data 2 times
  - Parity bit.
  - CRC
- Error correction

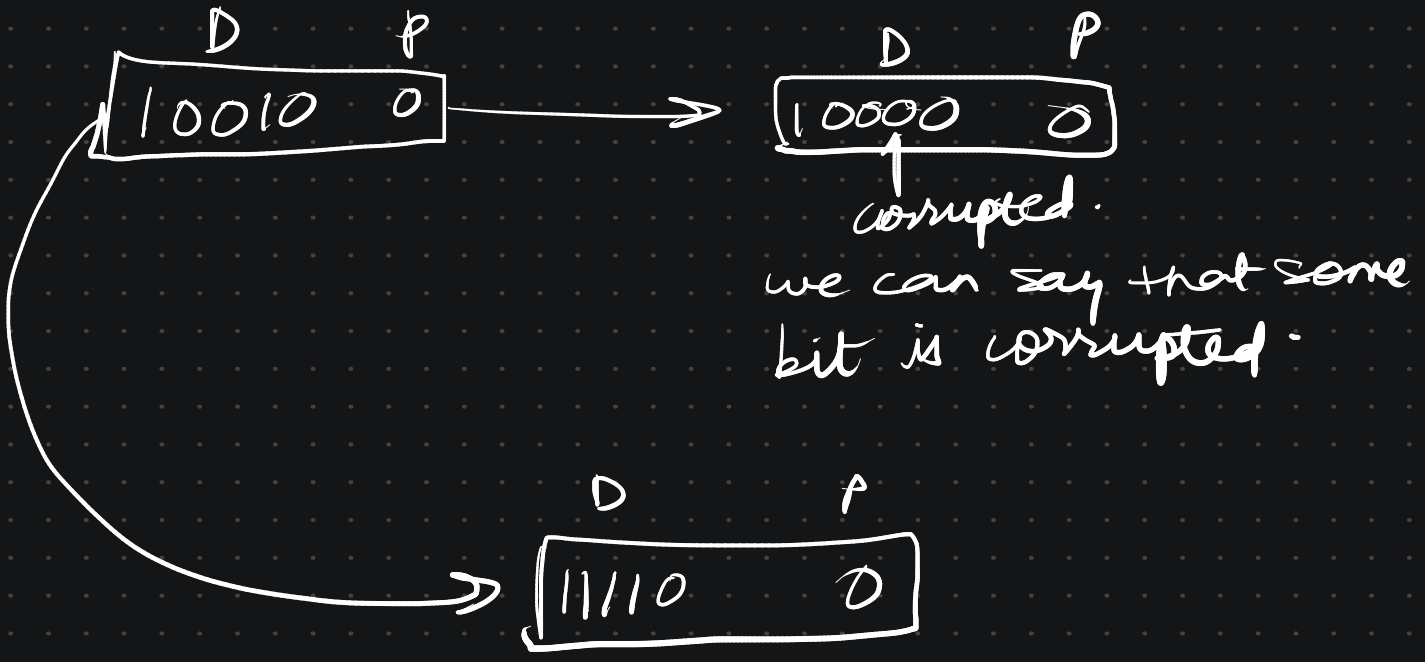
duplicate data for error detection:

If both copies are diff., then there's an error. Then request retransmission.

This method is not used. (Too much traffic)

There are error correction methods like Hamming code, but these are too resource intensive. So these aren't actually used in CNs.

Parity bit → Total 1's in packet are even → 0



we can say that some bit is corrupted.

we can't detect corruption

So this method is useless.

---

crc: cyclic redundancy check

used in CNs.
DLL uses it.

Eg. let Data = 1011011, CRC generator = 1011.
what is the data transmitted?

→ 1011 → 4 bits. CRC = 4 - 1 = 3 bits.

⊕ → modulo 2 sum
XOR

e.g. 1⊕0 = (1+0) %2
= 1 %2 = 1

1⊕1⊕1⊕1 = 4 %2 = 0

Data | 000 ← add 3 zeros for CRC

1011011 | 000

```
1101 ) 1011011000
     ⊕ 1101                ← Do XOR every time
       ----
       0110011000
     ⊕ 1101
       ----
       0001111000
     ⊕ 1101
       ----
       0011100
     ⊕ 1101
       ----
       0001   ← CRC is last 3 bits.
```

So data transmitted will be

1011011001

(001 = CRC)

sender sends 10110 1001

Data sent

Say receiver gets 11110 1001 ← corrupted

Receiver checks:

at the receiver end, the division with crc gen ought to result in a zero

```
1101 | 11110 1001
     ⊕ 1101
     ---------
       10011 001
     ⊕ 1101
     ---------
       10010 01
     ⊕ 1101
     ---------
       10000 1
     ⊕ 1101
     ---------
       1010 1
     ⊕ 1101
     ---------
       111 1
     ⊕ 1101
     ---------
       010
```

Not 0000 → 010

∴ Error in transmission.

CRC may get corrupted as well.

Q. Data = 1100111, CRC generator = 1011, CRC = ?

→ CRC = 3 bits

$$1011 \overline{)\,1100111000}$$

$$\oplus\ 1011$$

$$1111111000$$

$$\oplus\ 1011$$

$$1001100\,0$$

$$\oplus\ 1011$$

$$101000$$

$$\oplus\ 1011$$

$$\boxed{100} \leftarrow \text{CRC}$$

Data sent: $\boxed{1100111100}$

---

in the exam, CRC generator polynomial will be given

say $x^3 + x + 1$

↳ $1 \cdot x^3 + 0 \cdot x^2 + 1 \cdot x + 1 \cdot x^0$

$\boxed{1011} \leftarrow$ CRC generator

crc gen bits will be coefficients of decreasing powers of x

Q. Say data = 11010, CRC poly = x3 + x + 1, CRC = ?

→ CRC gen = 1011, CRC = 3 bits

```
1011 | 11010000
     ⊕1011
     -------
      1100000
      1011
      ------
       111000
       1011
       ------
        10100
        1011
        ----
          10
```

CRC = 010

There's a standard CRC generator (32-degree polynomial) that everyone uses.
Size of CRC generator will be 33
(powers of x^32, x^31, ..., x^0.
CRC will have 33-1 = 32 bits.

---

if crc gen is k bits, we add k-1 zeros
crc will be k-1 bits

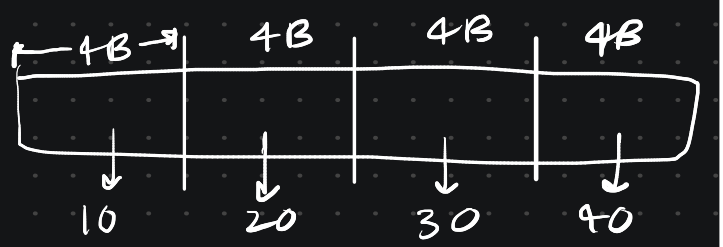
at receiver's,
we don't add any zeros

if CRC generator polynomial is k-degree,
then crc gen = k+1 bits, crc = k bits

Checksum:

used in TCP and IP.

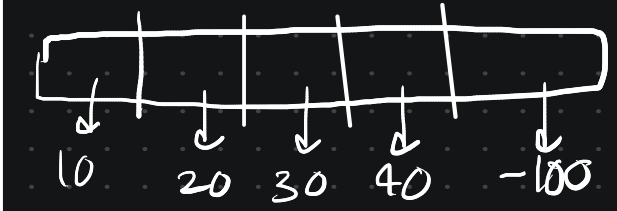
Data is divided into parts (4B)



10+20+30+40 = 100

checksum = -100.

At receiver's, CS



Add all.

you get 0 → no error

otherwise → error.

CS is also 32 bits. (4B)

Adding 32 bits to 32 bits would give us 32 bits result. plus carry

we add carry bit back to LSB.

↳ wrap-around carry. So checksum will be 32 bits, not 33.

With 32 bits, 2^32 combinations are possible. With n bits, 2^n combinations.
Two or more data would share same CS. Say D1, D100 share CS1.
If D1 gets corrupted and becomes D100, then the checksum would say data is fine.
So the methods don't fully guarantee 100% error detection.
But the prob. of such a situation is too low.

ISO/ OSI:

International Standards Organization/ Open Systems Interconnection

functions in CN → responsibilities of both Sw and H/w

- mandatory
  - → error control
  - → flow control
  - → access control
  - → multiplexing & demultiplexing
  - → addressing
  - ⋮
- optional
  - → encryption/decryption
  - → checkpointing
  - → routing

---

there are routing tables to decide which interface to send incoming packets to.

we can use CNs without routing as well.
If there aren't any routing tables here, then Flooding will be used in place of routing.
If we don't know where to send packets, the packets will be sent on all links.

---

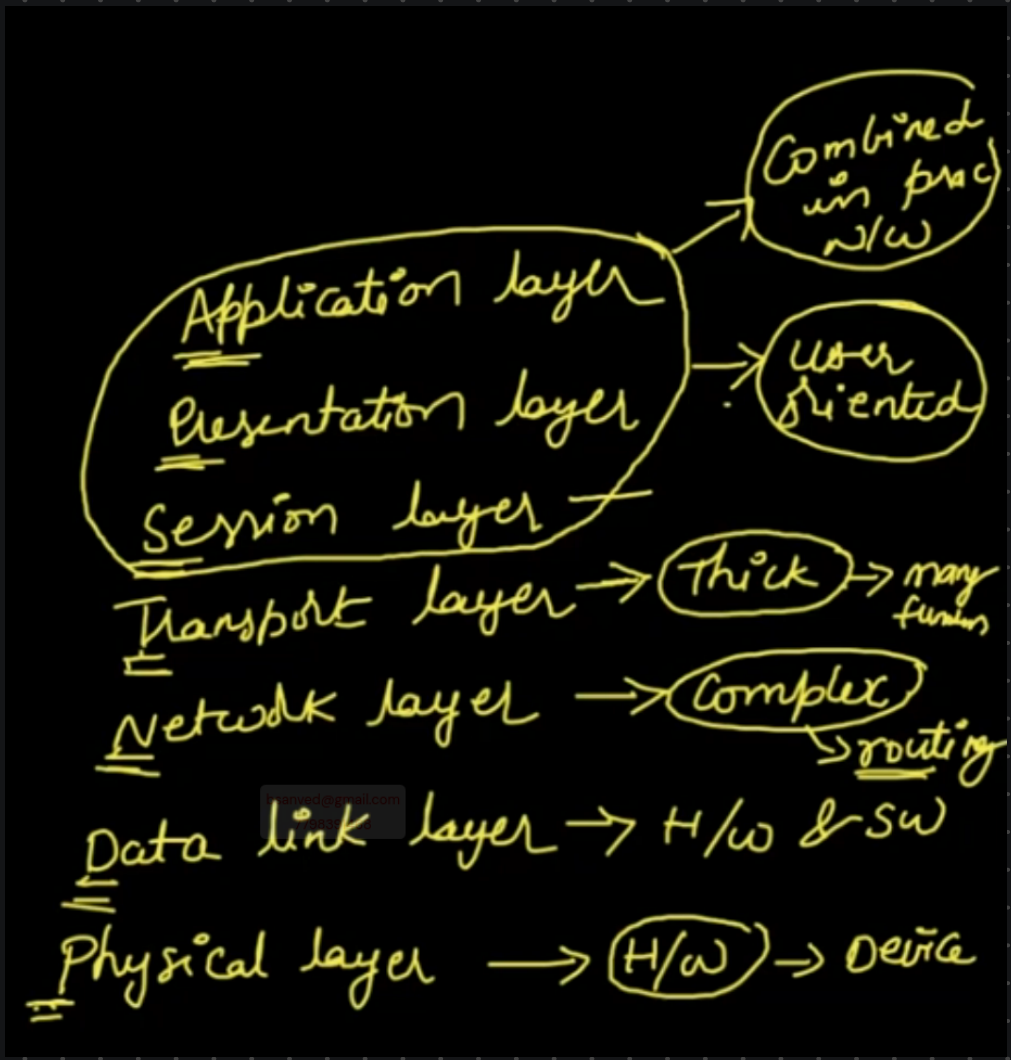
checkpointing: Say we're downloading a huge file. Checkpointing will be done periodically. If the connection breaks at a point, then the download will resume from the last checkpoint.

There are nearly 70ish functions in CN. We need to design software.

S/W
- monolith
- microkernel
- layered ← CN follows this

A layer has several functionalities. It can communicate with the layers above and below it.
We group all functions into various layers.
There are various groupings possible.
So there are various models:
- ISO-OSI (theoretical model)
- TCP-IP (practical model)
- ATM
- X.25
- IEEE

We'll discuss ISO-OSI (7 layers) and TCP-IP (5 layers).



ISO-OSI Model

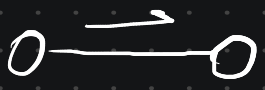
- Application Layer
- Presentation Layer
- Session Layer
- Transport Layer
- Network Layer
- Data Link Layer
- Physical Layer

Physical Layer:

- deals with electrical, mechanical, functional, procedural characteristics of physical links

Transmission Modes:

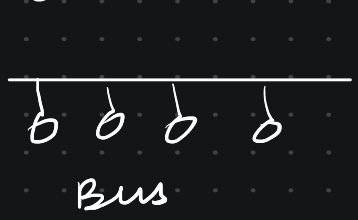
- Simplex: one-way
(antenna to television)

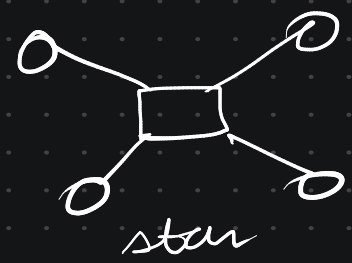


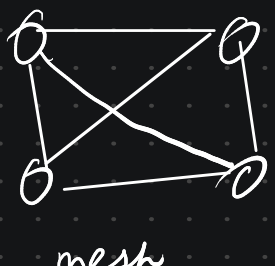
- Half-Duplex: both ways, but one at a time
(handheld transceiver)

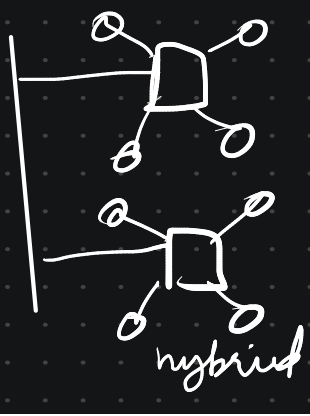
- Full Duplex: both ways, simultaneously
(phone n/w's)

Topologies:



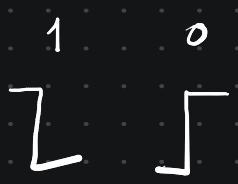




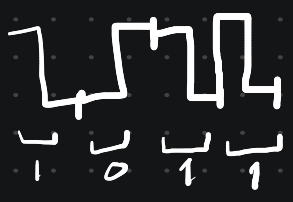


Encoding:

Manchester encoding:



1011 →



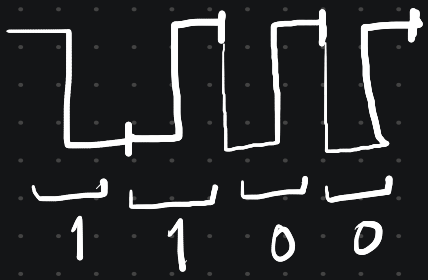
0111 →

Differential Manchester Encoding:

(vertical line at the start means 0)

we get 2 diff. waves.

Say 1100

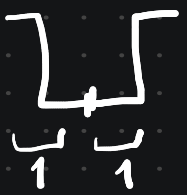


choose that version of the bit that results in vertical line if we want 0.

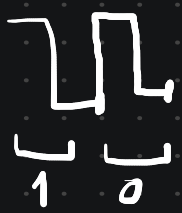
For 1, choose that option that gives no vertical line.

Say we want 1. Choose either.

Now for another one, we want no line |

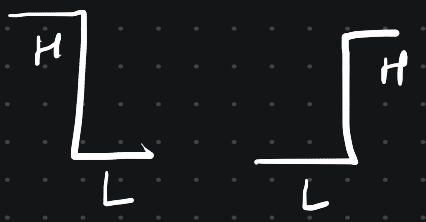


Instead going up would give 10



The first choice would determine the rest (for a given sequence).

---

1 bit in Manchester: 

For 1 bit, we use 2 voltages.

In differential:

1 bit → 2 voltages

so to send say 100 bits, we need to change voltage 200 times.

To send 100 bps, we need to change voltage 200 times/sec

That's the Baud rate: voltages/sec

Baud Rate = 2 * Bit Rate